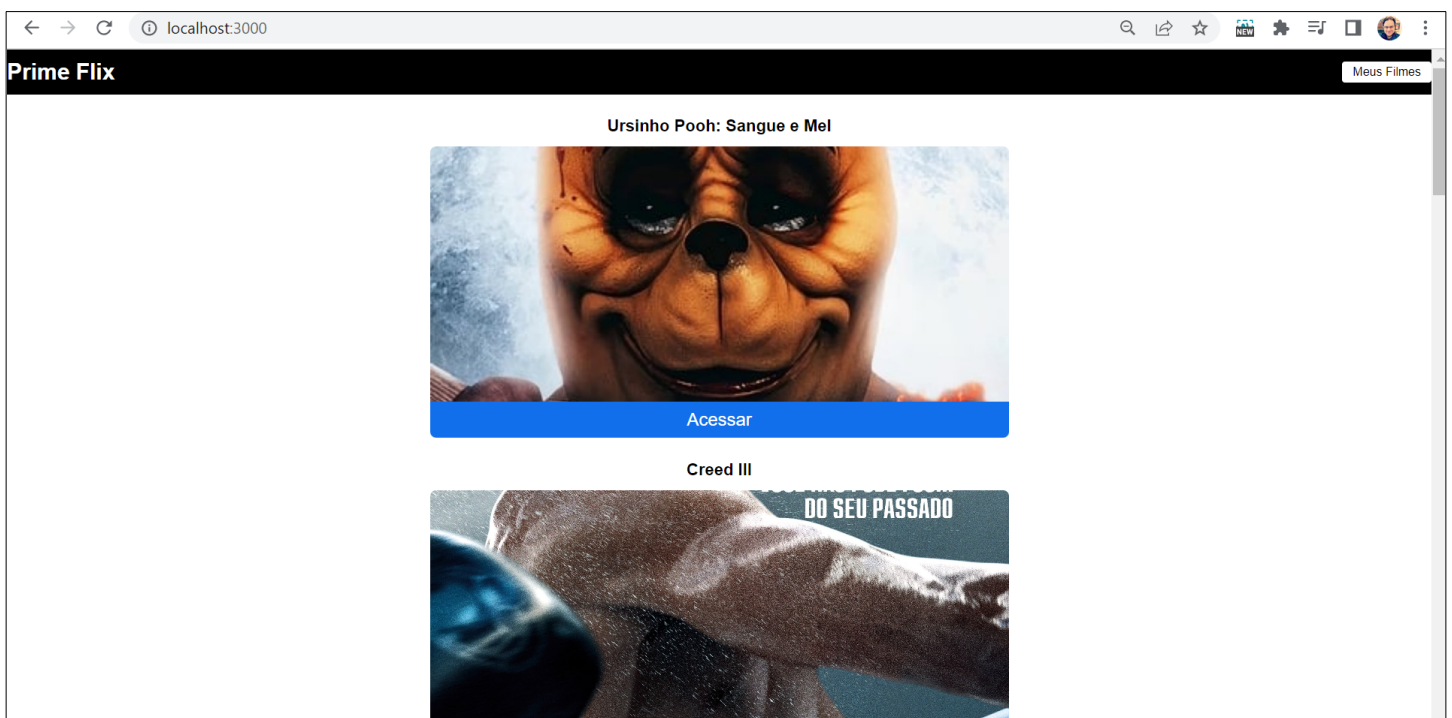


Aula 5 – Prime Flix

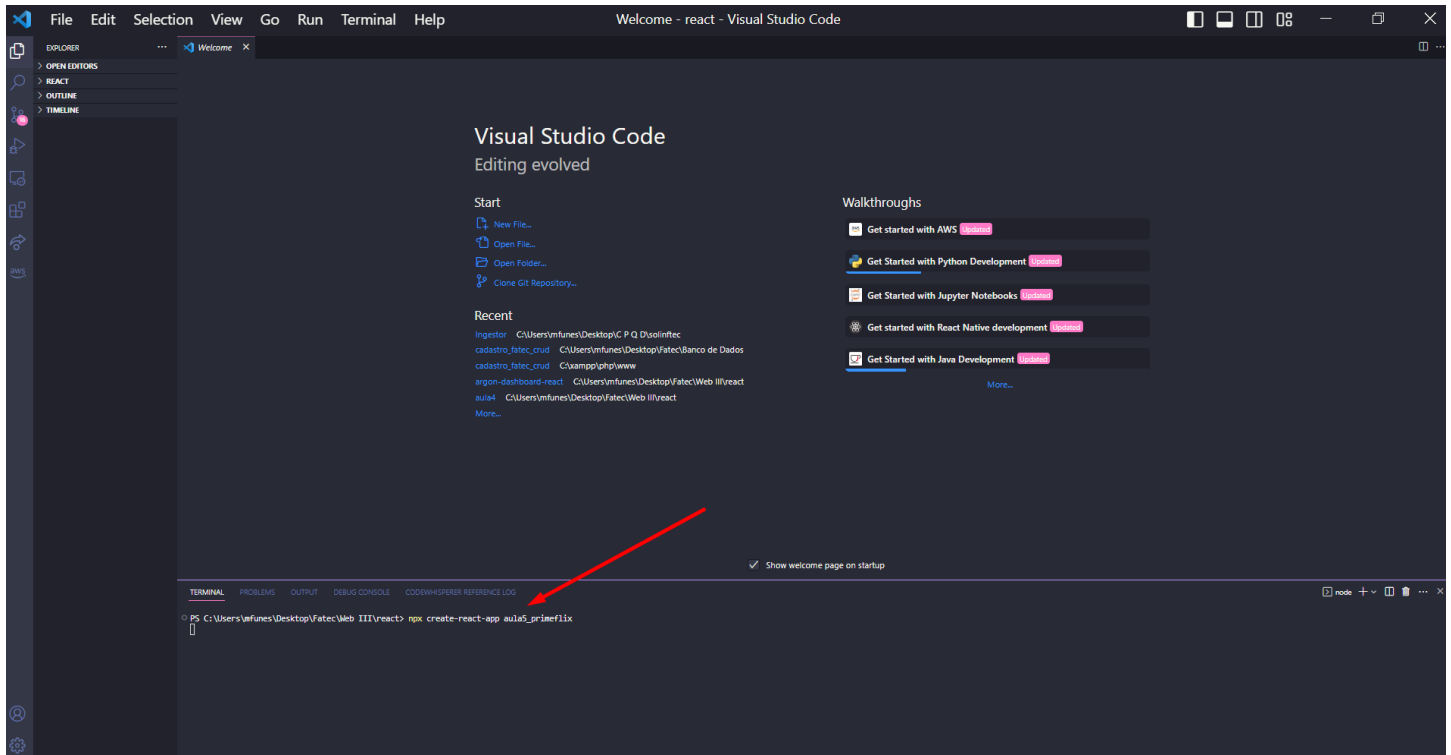
Objetivo da aula:

- Criar uma aplicação chamada Prime Flix
- Consumir API de filmes
- Configurar estados e renderização de páginas
- Configurar rotas dinâmicas com base no retorno da API
- Montar a Home listando filmes requisitados via API

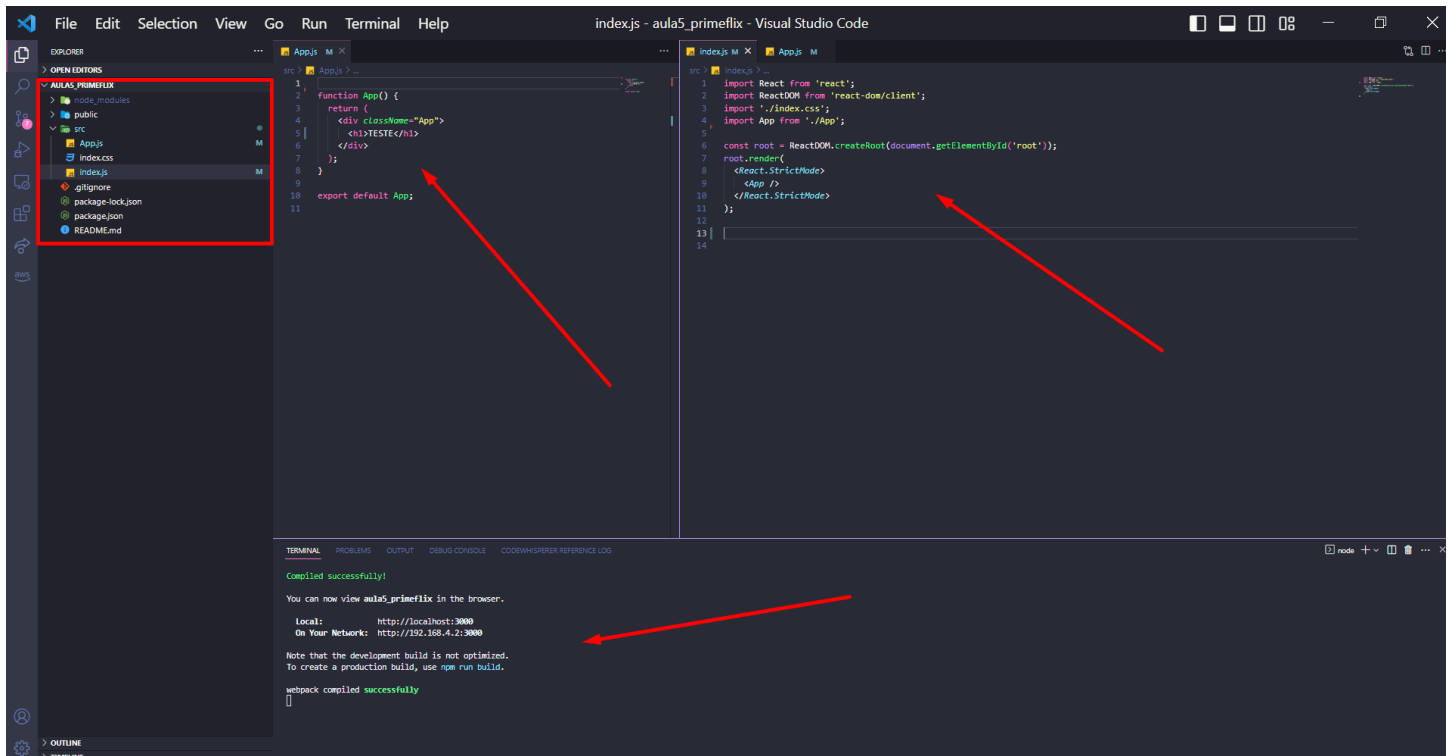


Setup básico do projeto React

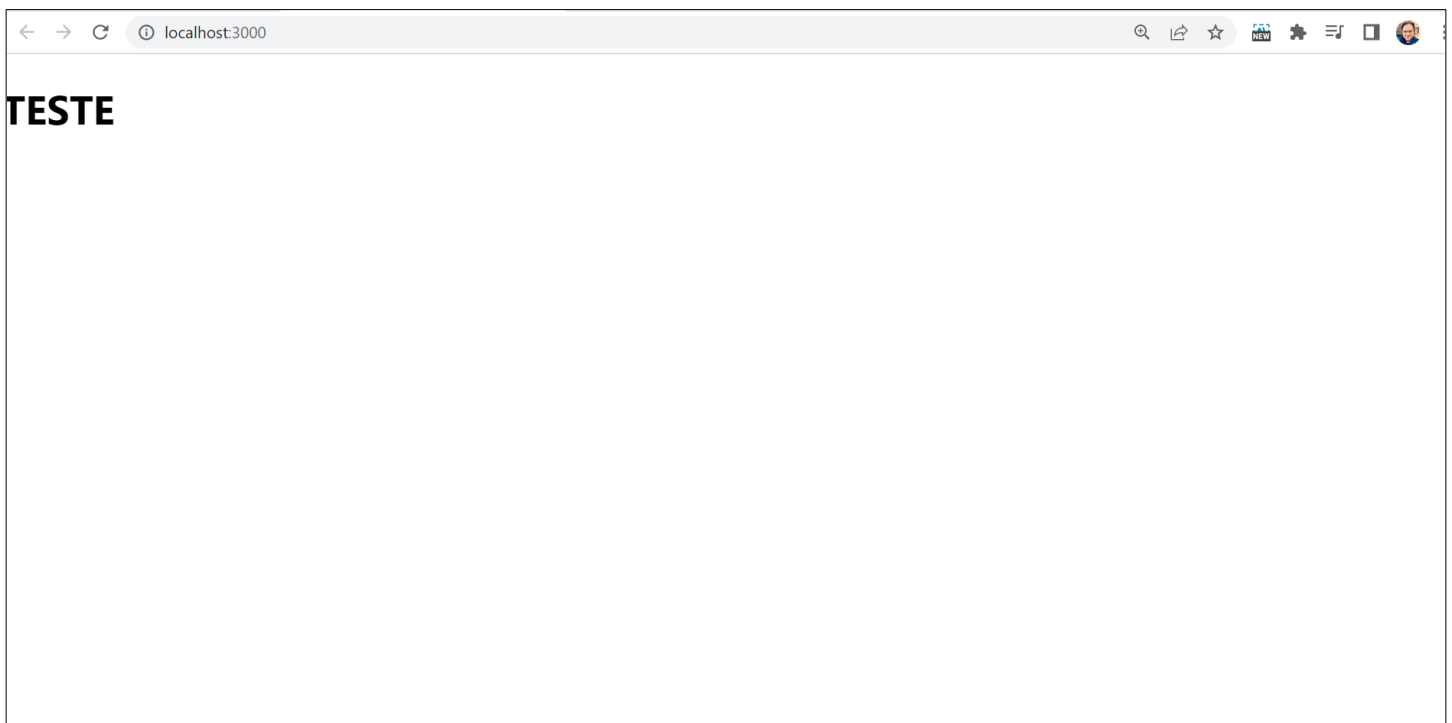
1 – Crie um novo projeto chamado primeFlix (ou o nome que desejar)



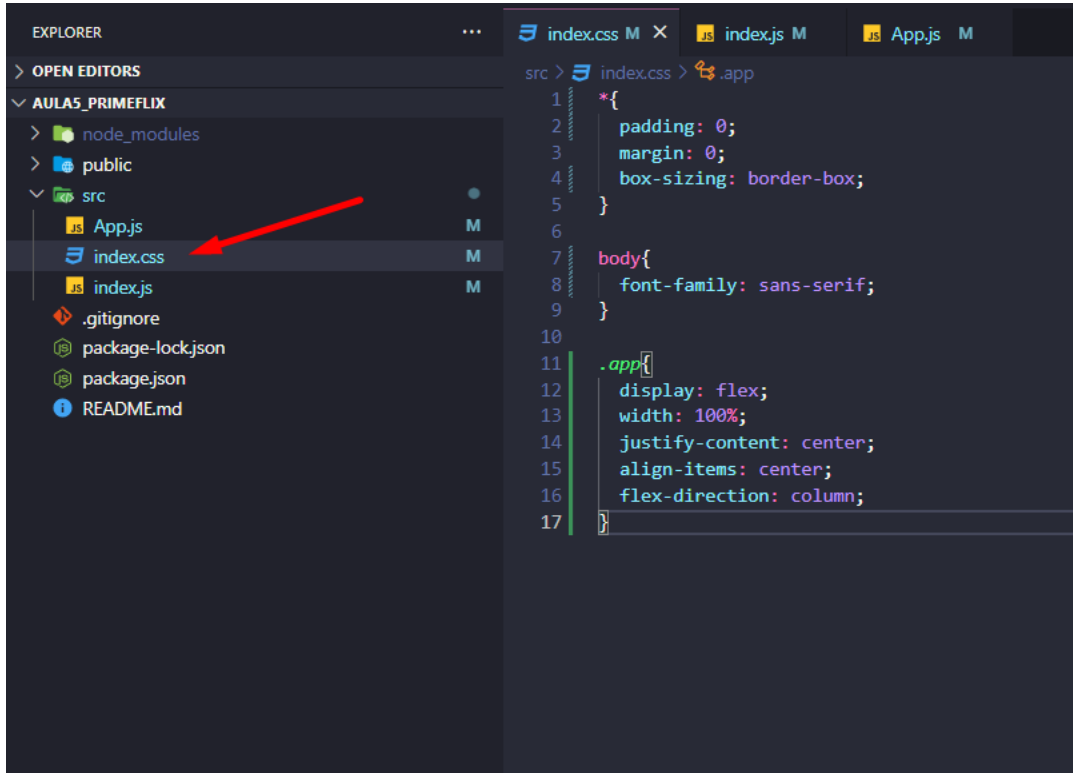
2 – Após deixarmos o projeto enxuto, ele terá essa aparência:



3 – Inicie seu projeto, ele deverá ter essa aparência:

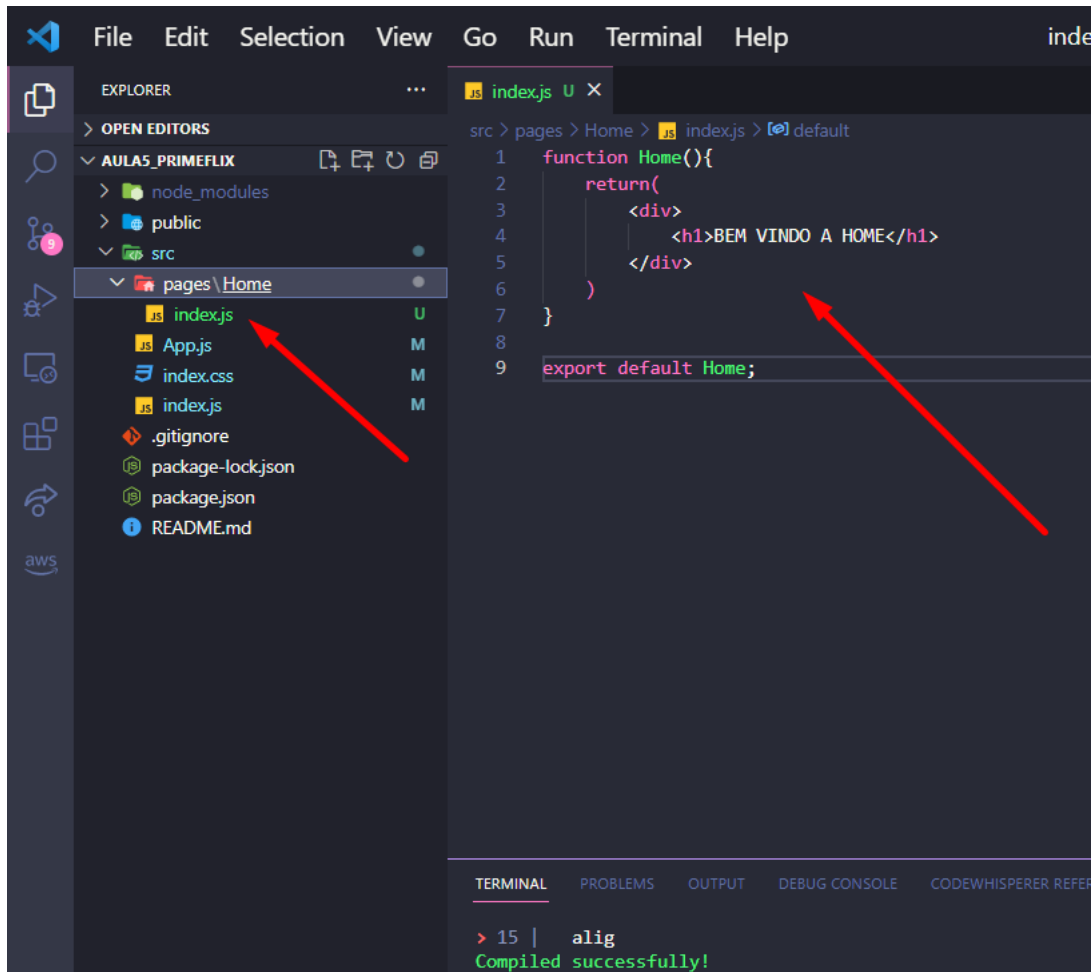


4 -Faça uma configuração padrão para o css da nossa aplicação. O CSS abaixo funciona bem como base para a maioria das aplicações web.

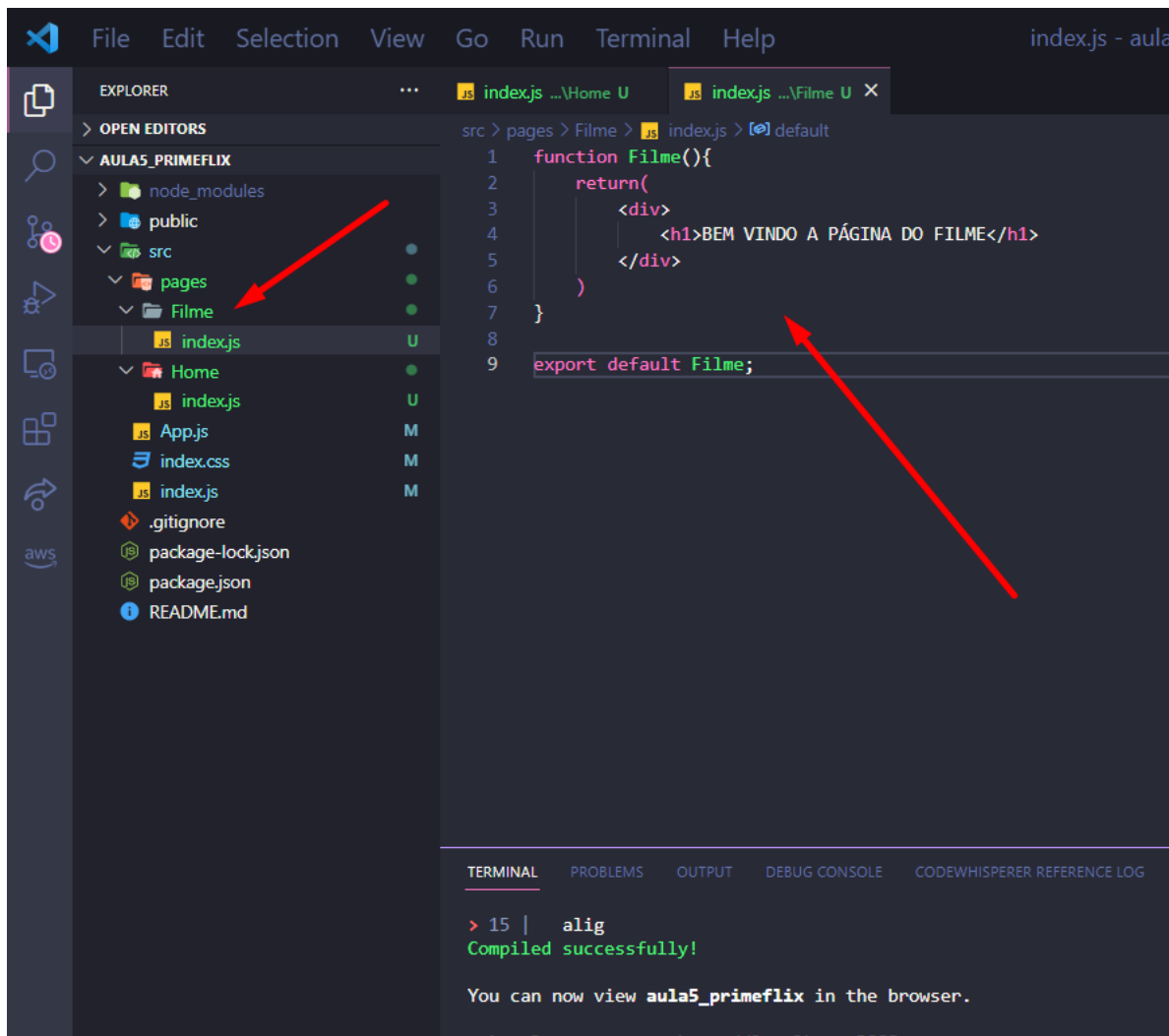


Configurando nossas rotas

5 – Dentro de SCR crie uma pasta chamada pages, dentro de pages crie outra pasta chamada Home. Dentro da pasta Home crie um arquivo chamado index.js, essa será a página principal do projeto.



6 – Crie uma nova página dentro de pages chamada Filme, dentro de Filme crie um arquivo chamado index.js, como demonstrado abaixo:



7 - Pare de rodar o projeto (utilize CTRL + C) e vamos instalar o componente react-router-dom para configurar nossas rotas.

```
function Filme(){
  return(
    <div>
      <h1>BEM VINDO A PÁGINA DO FILME</h1>
    </div>
  )
}

export default Filme;
```

Compiled successfully!

You can now view **aula5_primeflix** in the browser.

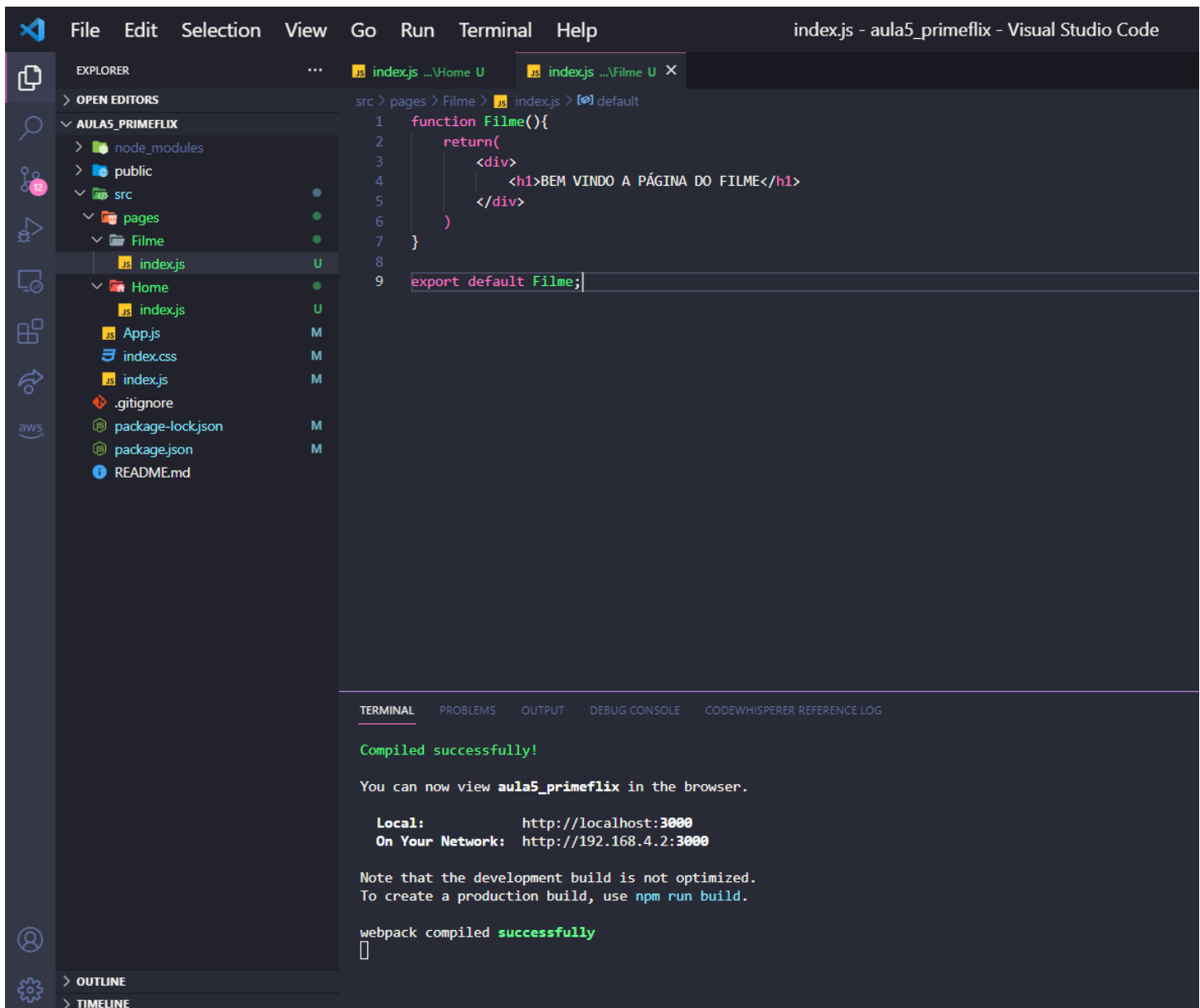
Local: http://localhost:3000
On Your Network: http://192.168.4.2:3000

Note that the development build is not optimized.
To create a production build, use `npm run build`.

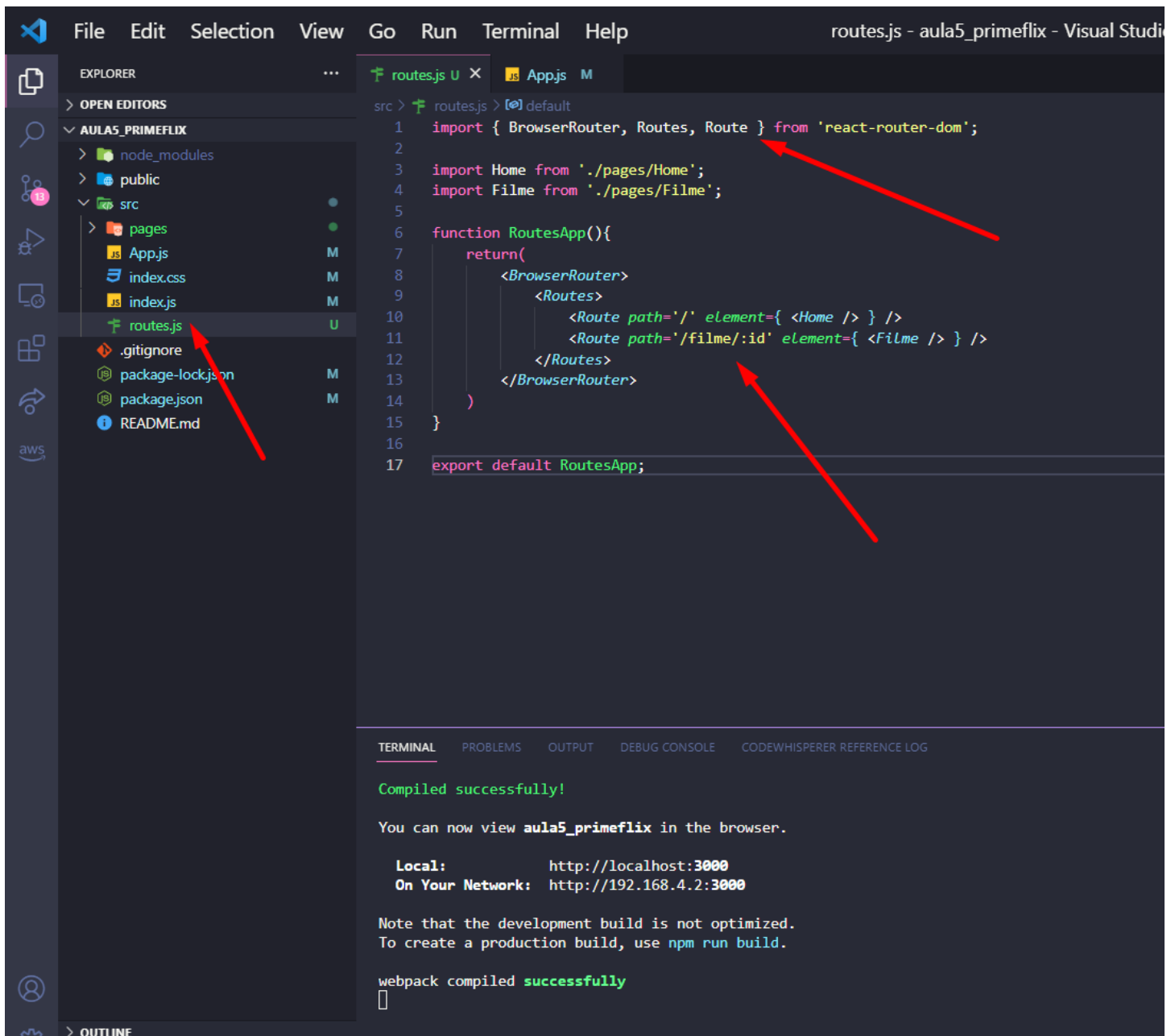
webpack compiled **successfully**
Deseja finalizar o arquivo em lotes (S/N)? s

PS C:\Users\mfunes\Desktop\Fatec\Web III\react\aula5_primeflix> npm install react-router-dom

8 – Rode novamente o projeto (npm start) e veja se está tudo ok.



9 – Vamos agora de fato criar nossas rotas. Crie um arquivo dentro de SRC chamado routes.js. Faça a configuração das rotas seguindo o exemplo abaixo. Veja que agora Home é a rota padrão "/" e Filme tem a rota /filme/, inclusive já deixamos um .id para, via API, poder mostrar diversos filmes nessa mesma rota :D



The screenshot shows the Visual Studio Code interface with the 'routes.js' file open in the editor. The Explorer sidebar on the left shows the project structure, with 'routes.js' highlighted in the 'src' directory. The editor displays the following code:

```
src > routes.js > [default]
1  import { BrowserRouter, Routes, Route } from 'react-router-dom';
2
3  import Home from './pages/Home';
4  import Filme from './pages/Filme';
5
6  function RoutesApp(){
7    return(
8      <BrowserRouter>
9        <Routes>
10         <Route path="/" element={ <Home /> } />
11         <Route path="/filme/:id" element={ <Filme /> } />
12       </Routes>
13     </BrowserRouter>
14   )
15 }
16
17 export default RoutesApp;
```

Three red arrows point to specific parts of the code: one to the import statement for 'react-router-dom', one to the 'element' prop in the first route, and one to the 'path' prop in the second route.

The terminal at the bottom shows the following output:

```
TERMINAL  PROBLEMS  OUTPUT  DEBUG CONSOLE  CODEWHISPERER REFERENCE LOG

Compiled successfully!

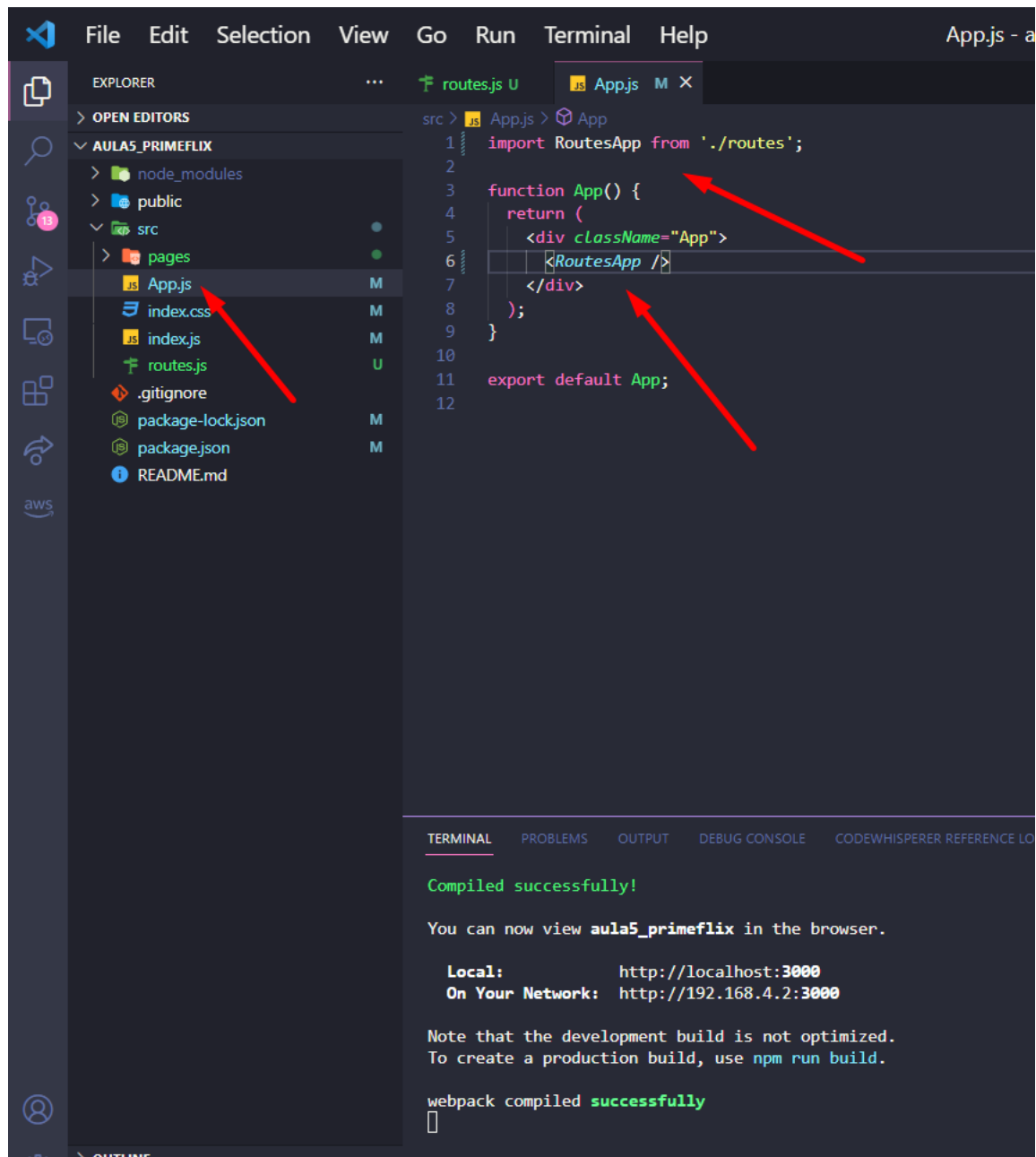
You can now view aula5_primeflix in the browser.

Local:      http://localhost:3000
On Your Network: http://192.168.4.2:3000

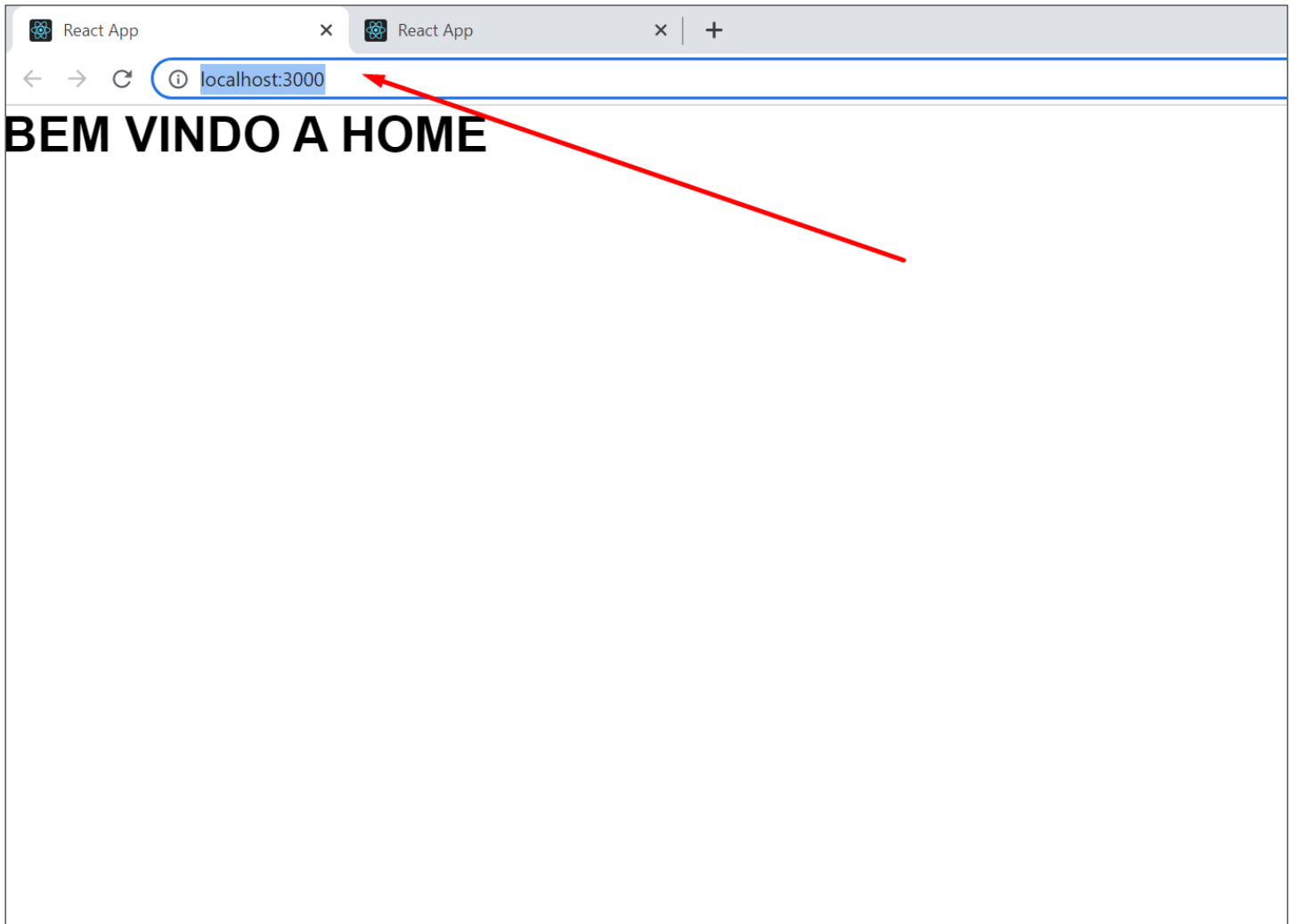
Note that the development build is not optimized.
To create a production build, use npm run build.

webpack compiled successfully
```

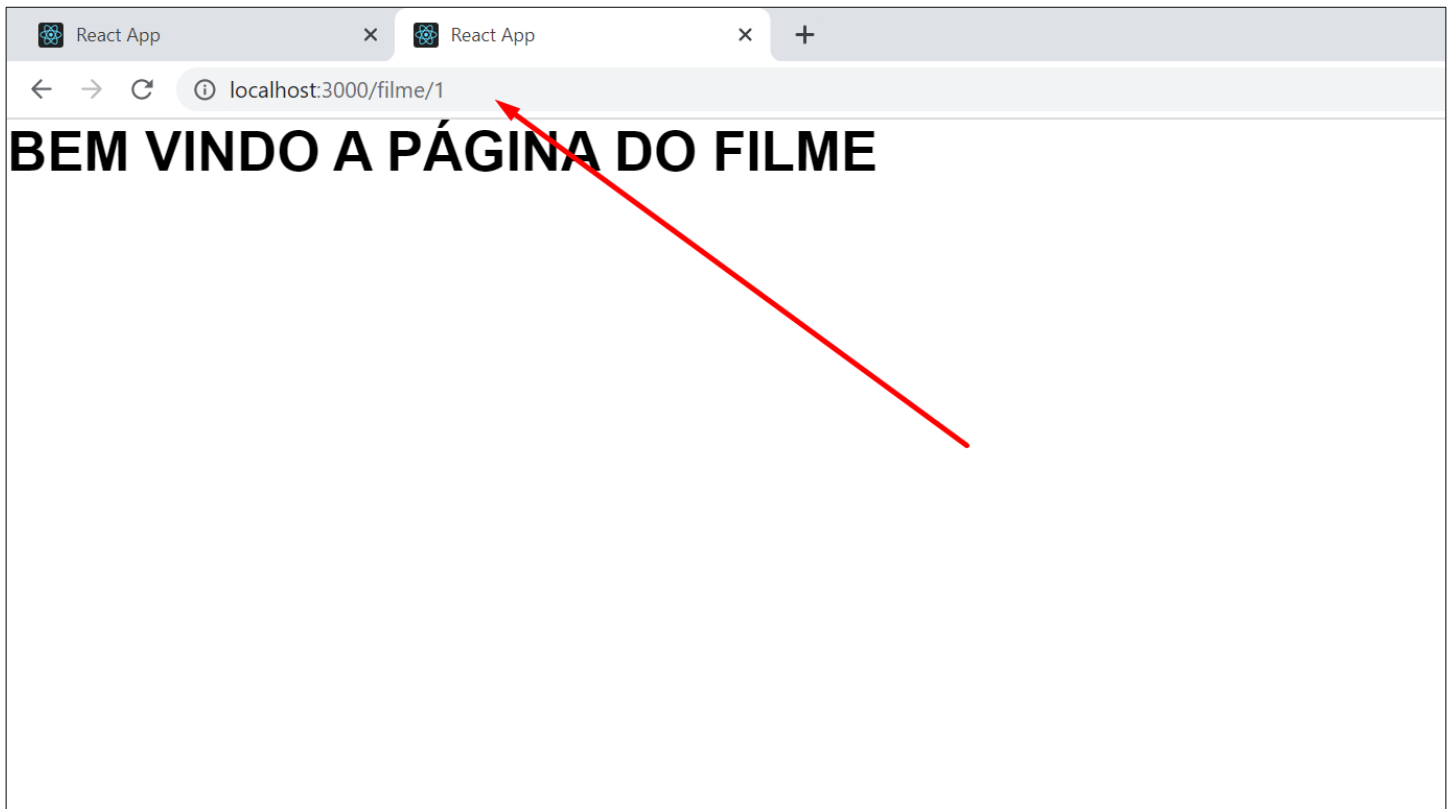
10 – Vá até APP.js e importe nosso componente RoutesAPP, dentro da <div> vamos renderizar nosso componente RoutesAPP que possui a configuração de rotas.



11 – Faça o testes da rotas, ao entrar no site ele deverá mostrar a página Home

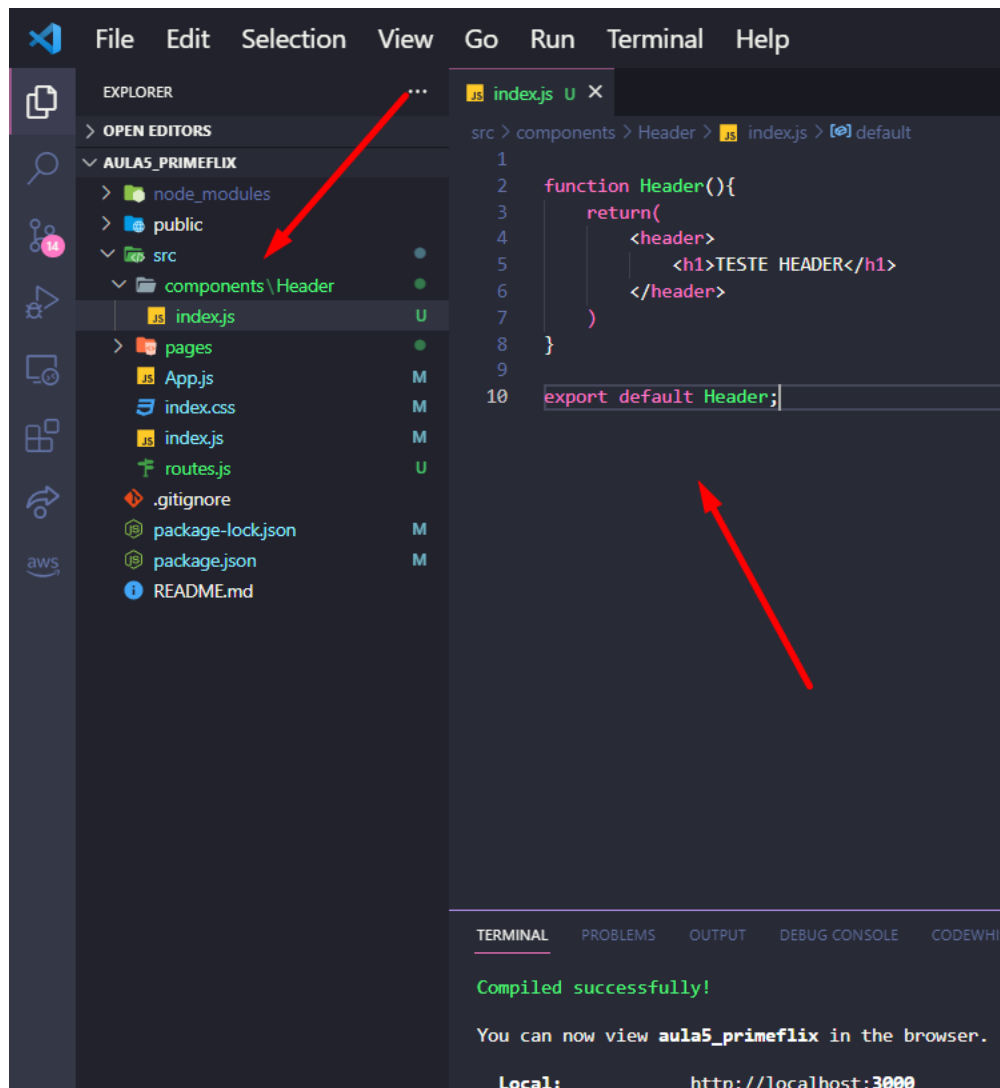


12 – Ao colocarmos a rota configurada /filme/1 (aqui para testar colocamos qualquer id) ele deverá mostrar a página Filme.



Criando um HEADER comum a todas as páginas

13 – Vamos agora criar um novo componente, como ele será reutilizado por várias página, crie uma pasta dentro de SRC chamada “components”, aqui você irá colocar todos os componentes que serão reutilizados. Dentro da pasta components crie uma pasta chamada Header e dentro de Header um arquivo index.js. Sempre que precisar criar um componente no React faça estes passos. Crie a função Header dentro de index.js como abaixo:



The screenshot shows the Visual Studio Code interface. On the left, the Explorer sidebar displays the project structure. A red arrow points to the 'components' folder under the 'src' directory, which has been expanded to show a sub-folder named 'Header'. Inside 'Header', a file named 'index.js' is listed. The main editor area on the right shows the content of 'index.js'. It contains a function definition for 'Header' that returns a JSX element with a heading 'TESTE HEADER'. Below the function, the 'Header' component is exported as the default export. A second red arrow points to the 'export default Header;' line. At the bottom, the Terminal panel shows a successful compilation message and the local development URL: 'http://localhost:3000'.

```
File Edit Selection View Go Run Terminal Help

EXPLORER
> OPEN EDITORS
AULAS_PRIMEFLIX
  > node_modules
  > public
  > src
    > components \Header
      index.js
    > pages
      App.js
      index.css
      index.js
      routes.js
    .gitignore
    package-lock.json
    package.json
    README.md

index.js
1
2 function Header(){
3   return(
4     <header>
5       <h1>TESTE HEADER</h1>
6     </header>
7   )
8 }
9
10 export default Header;
```

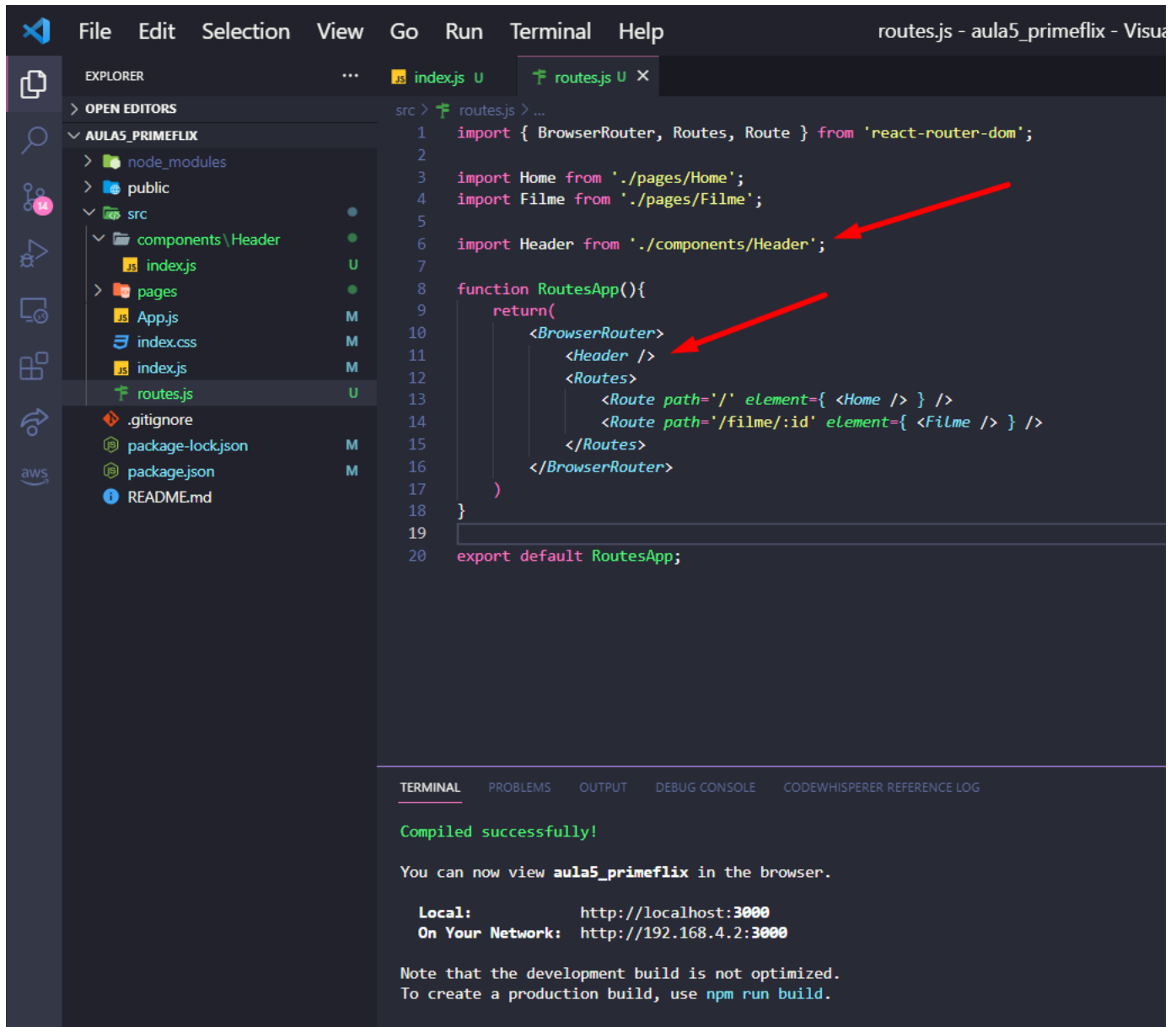
TERMINAL PROBLEMS OUTPUT DEBUG CONSOLE CODEWHISPERER

Compiled successfully!

You can now view **aula5_primeflix** in the browser.

Local: http://localhost:3000

14 – Vamos agora renderizar nosso novo componente Header, vá até routes.js, importe o componente Header e dentro do componente RoutesAPP coloque <Header /> desse modo o Header sempre será renderizado antes de qualquer página e independente da página que estiver sendo renderizada, lá estará o nosso reader.



The screenshot shows the Visual Studio Code editor with the file explorer on the left and the code editor on the right. The file explorer shows the project structure for 'aula5_primeflix', with the 'src' directory expanded to show 'components/Header' and 'pages'. The 'routes.js' file is selected in the file explorer and open in the editor. The code in 'routes.js' is as follows:

```
src > routes.js > ...
1  import { BrowserRouter, Routes, Route } from 'react-router-dom';
2
3  import Home from './pages/Home';
4  import Filme from './pages/Filme';
5
6  import Header from './components/Header';
7
8  function RoutesApp(){
9    return(
10     <BrowserRouter>
11       <Header />
12       <Routes>
13         <Route path="/" element={ <Home /> } />
14         <Route path="/filme/:id" element={ <Filme /> } />
15       </Routes>
16     </BrowserRouter>
17   )
18 }
19
20 export default RoutesApp;
```

Two red arrows point to the import statement on line 6 and the <Header /> component on line 11. The terminal at the bottom shows the following output:

```
TERMINAL  PROBLEMS  OUTPUT  DEBUG CONSOLE  CODEWHISPERER REFERENCE LOG

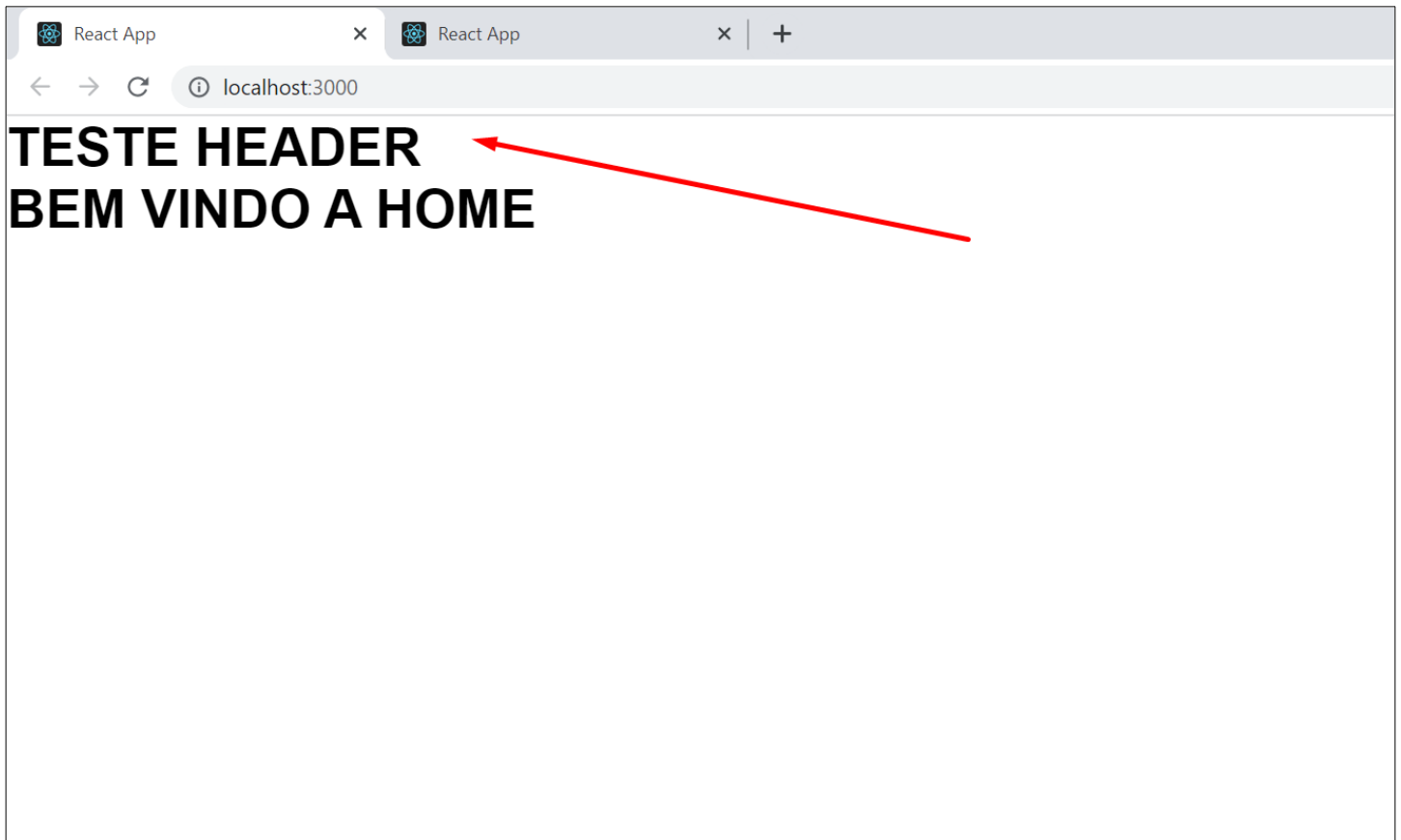
Compiled successfully!

You can now view aula5_primeflix in the browser.

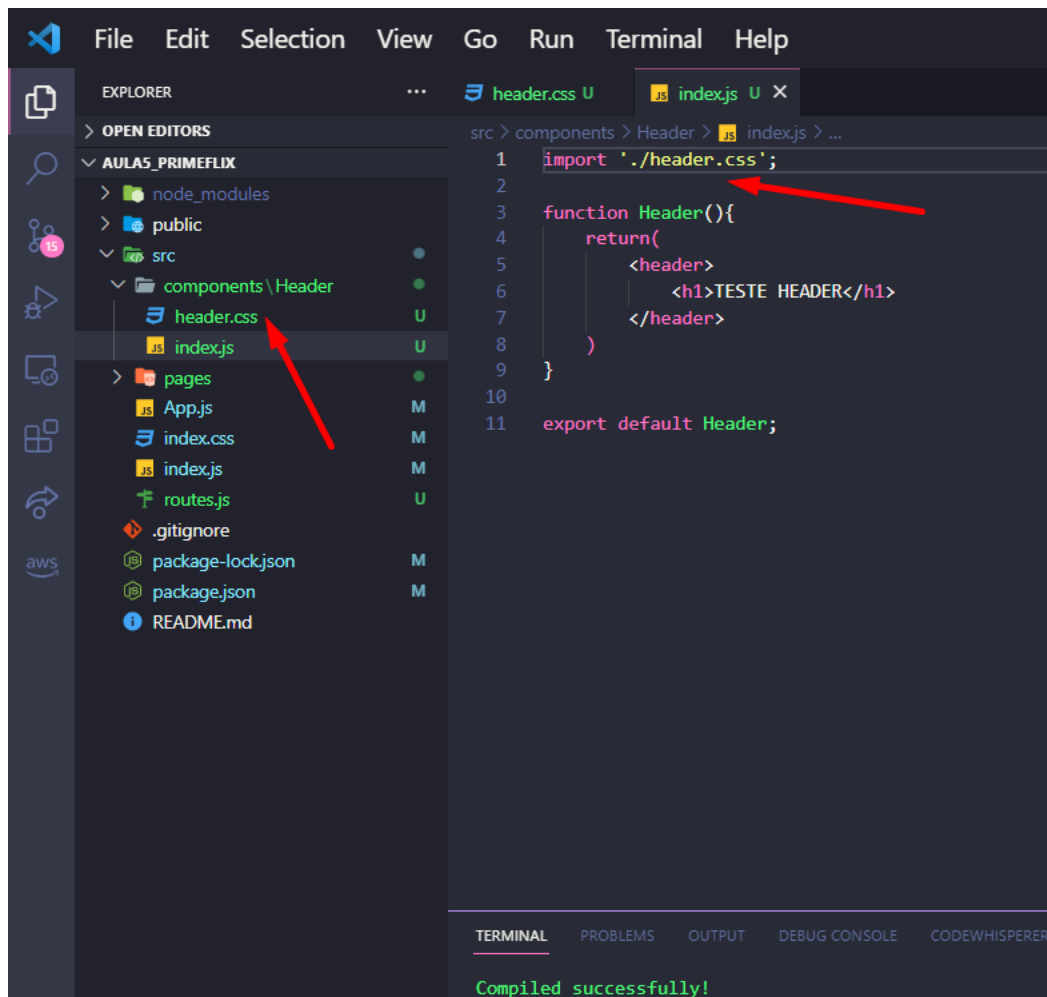
Local:      http://localhost:3000
On Your Network:  http://192.168.4.2:3000

Note that the development build is not optimized.
To create a production build, use npm run build.
```

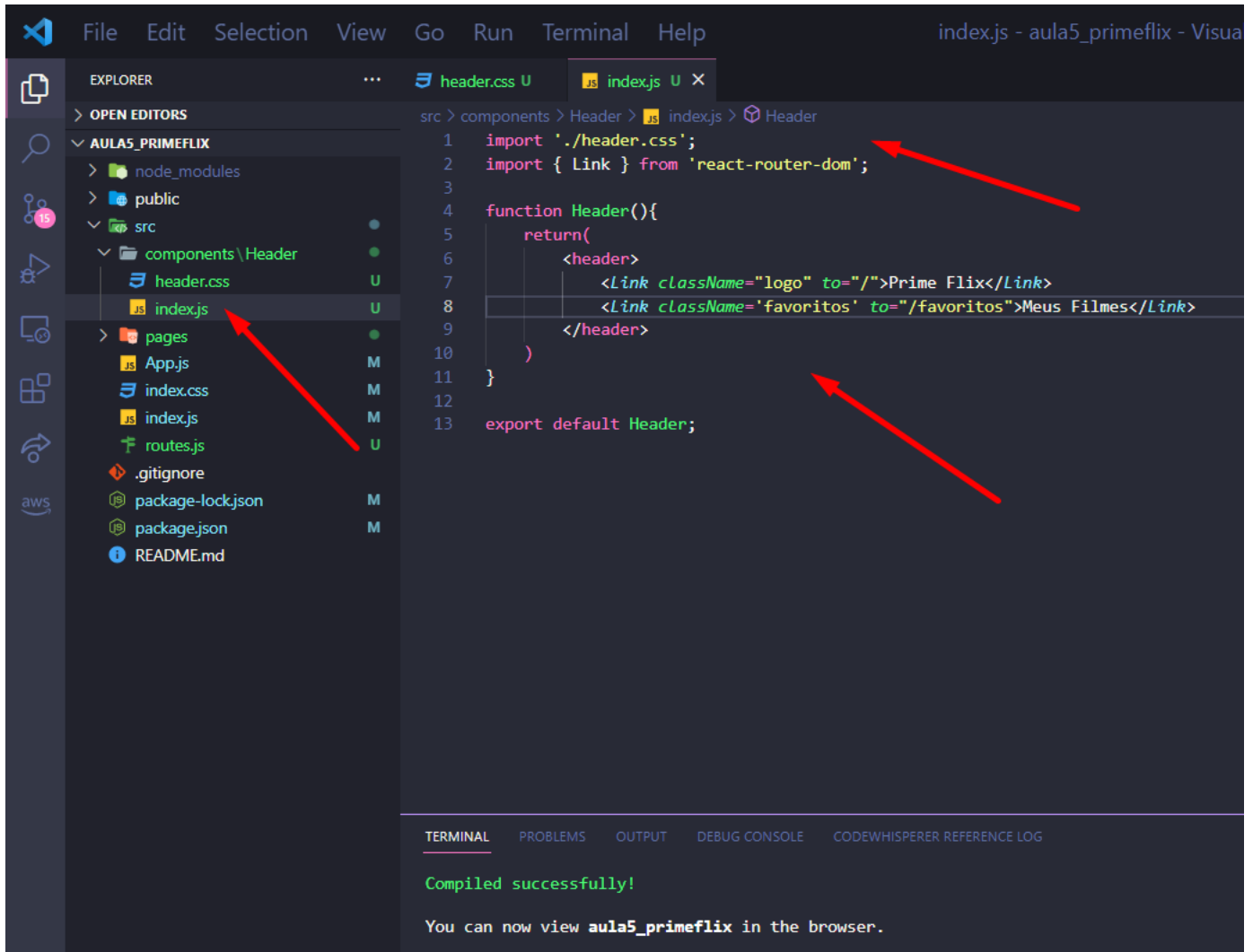
15 – Veja se esse é o resultado obtido com nossa configuração do Header. Vamos agora estilizar o Header e deixar ele mais bonito.



16 – Como queremos estilizar o componente Header, dentro da pasta Header crie um novo arquivo chamado header.css (aqui podemos colocar qualquer novo para o arquivo .css). Vá dentro do index.js responsável pelo componente Header e faça a importação do header.css.



17 – Aproveite que ainda estamos em index.js e faça as seguintes modificações. Importe o componente Link do react-router-dom. Também queremos que nosso Header tenha um link para a página Home “/” e já vamos criar um link para favoritos, esse componente e rota para favoritos nós ainda iremos criar.



```
File Edit Selection View Go Run Terminal Help index.js - aula5_primeflix - Visua
```

EXPLORER

OPEN EDITORS

AULAS_PRIMEFLIX

- node_modules
- public
- src
 - components \Header
 - header.css U
 - index.js U**
 - pages
 - App.js M
 - index.css M
 - index.js M
 - routes.js U
 - .gitignore
 - package-lock.json M
 - package.json M
 - README.md

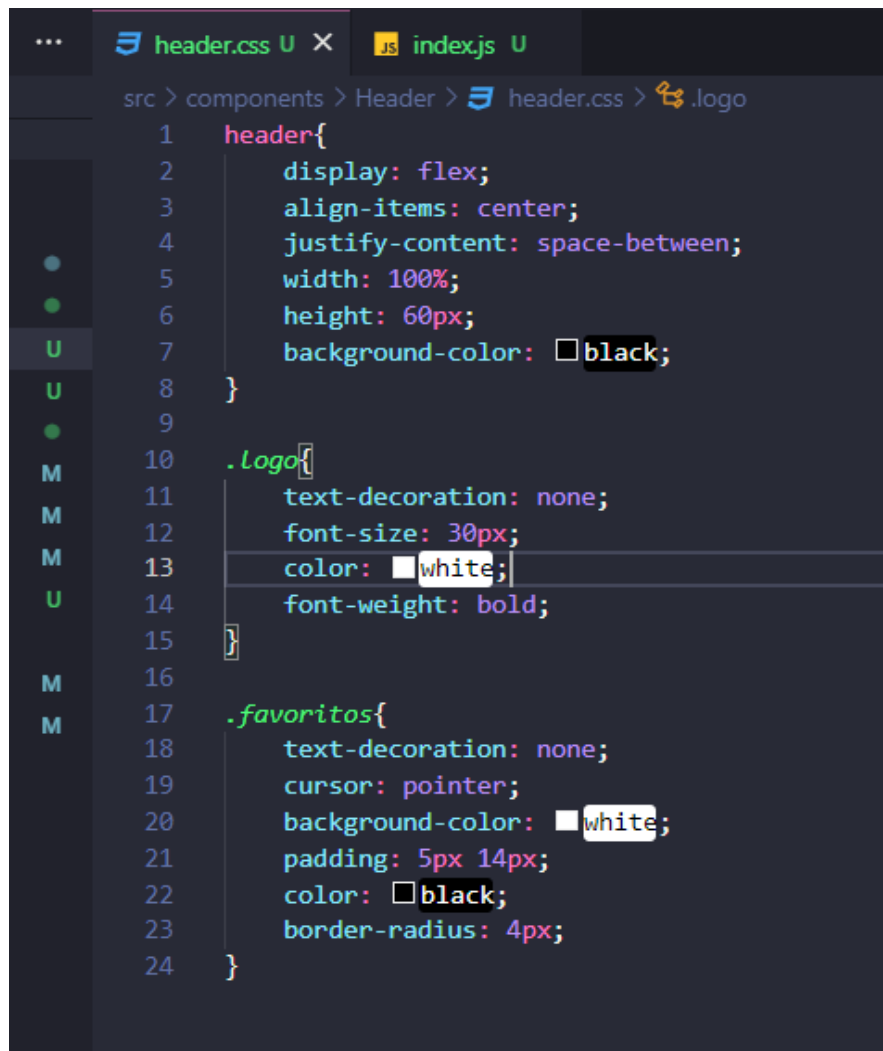
```
src > components > Header > index.js > Header
1  import './header.css';
2  import { Link } from 'react-router-dom';
3
4  function Header(){
5      return(
6          <header>
7              <Link className="logo" to="/">Prime Flix</Link>
8              <Link className='favoritos' to="/favoritos">Meus Filmes</Link>
9          </header>
10      )
11  }
12
13  export default Header;
```

TERMINAL PROBLEMS OUTPUT DEBUG CONSOLE CODEWHISPERER REFERENCE LOG

Compiled successfully!

You can now view `aula5_primeflix` in the browser.

18 – Aqui está uma sugestão de estilização do Header, caso queira pode criar seu próprio estilo.



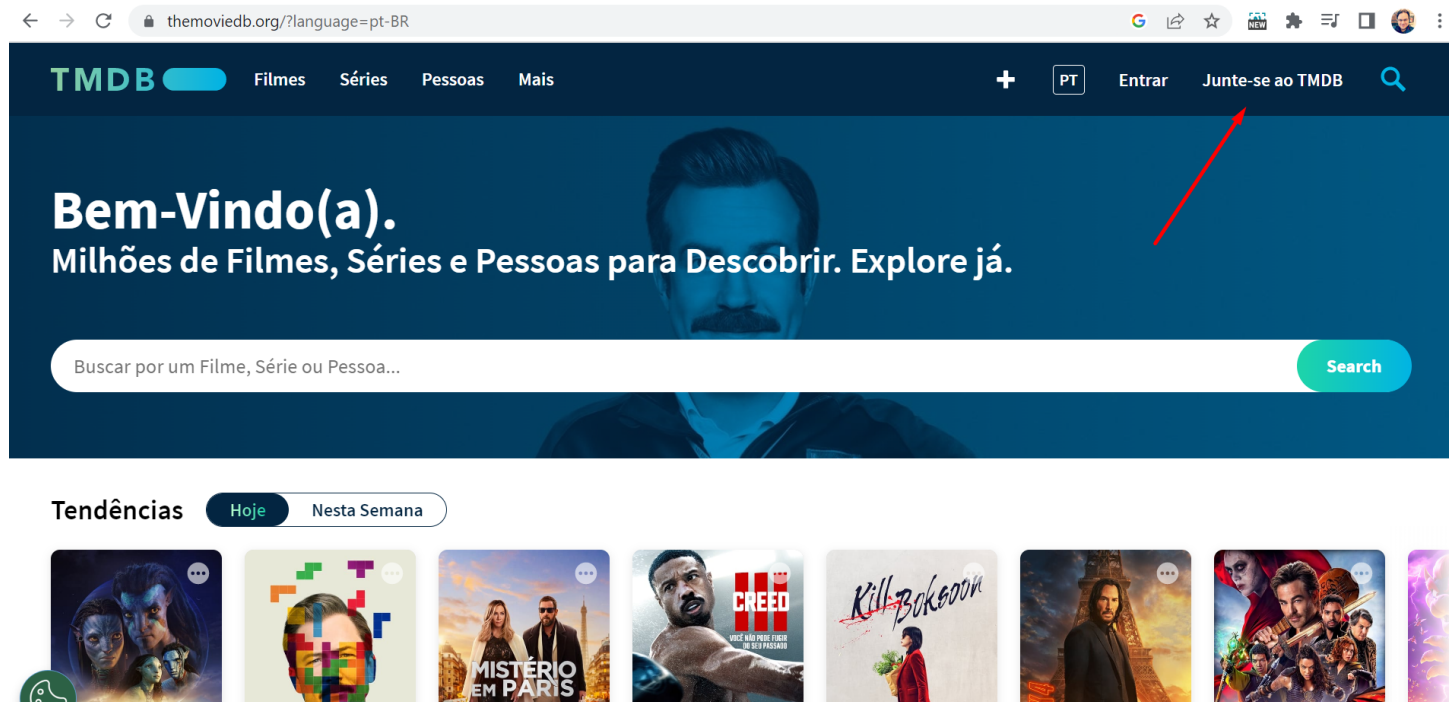
```
... header.css U X JS index.js U
src > components > Header > header.css > .logo
1  header{
2      display: flex;
3      align-items: center;
4      justify-content: space-between;
5      width: 100%;
6      height: 60px;
7      background-color: black;
8  }
9
10  .Logo{
11      text-decoration: none;
12      font-size: 30px;
13      color: white;
14      font-weight: bold;
15  }
16
17  .favoritos{
18      text-decoration: none;
19      cursor: pointer;
20      background-color: white;
21      padding: 5px 14px;
22      color: black;
23      border-radius: 4px;
24  }
```

19 – Sua aplicação deverá ter essa aparência. Veja que ao clicar em “Meus Filmes” ele irá para uma nova página (que ainda iremos criar, ok?) mas o Header é fixo pois estamos sempre renderizando ele acima de todas as páginas nas rotas.

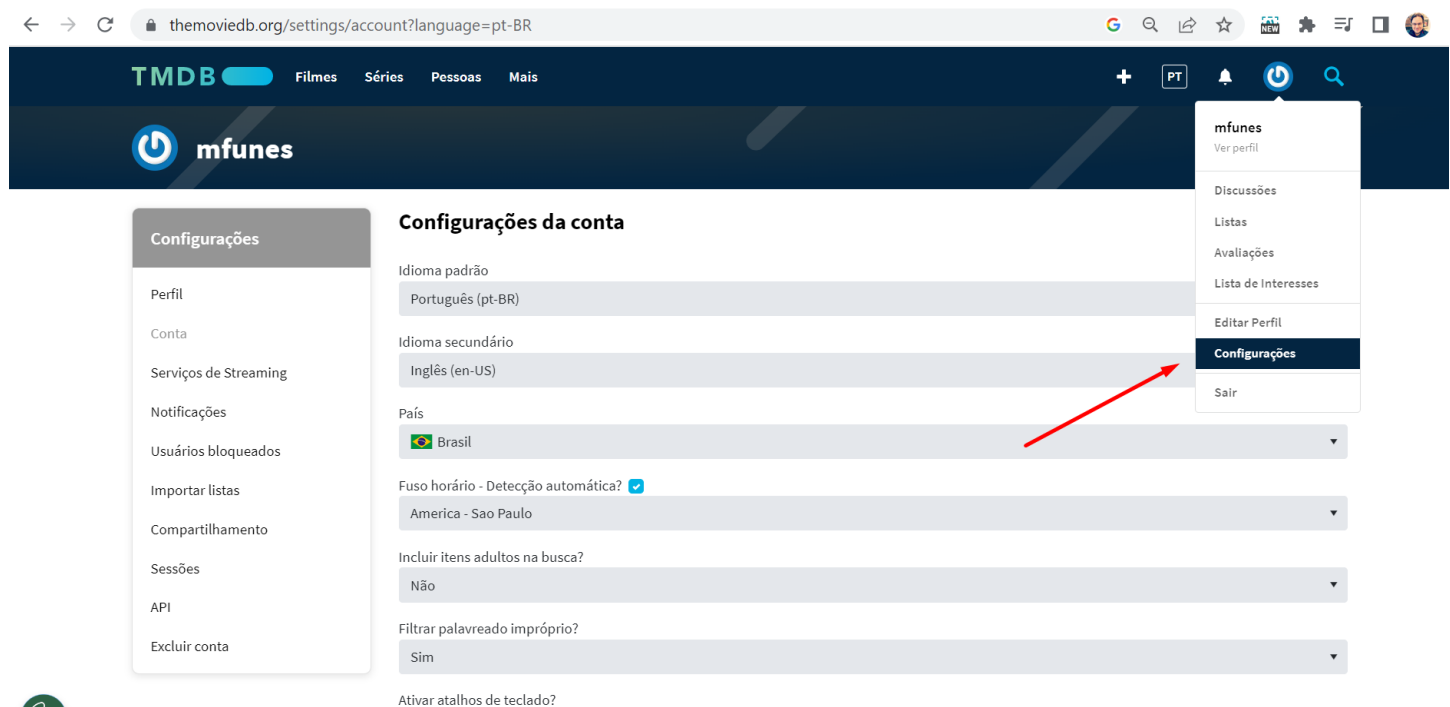


Configurando a Home via API

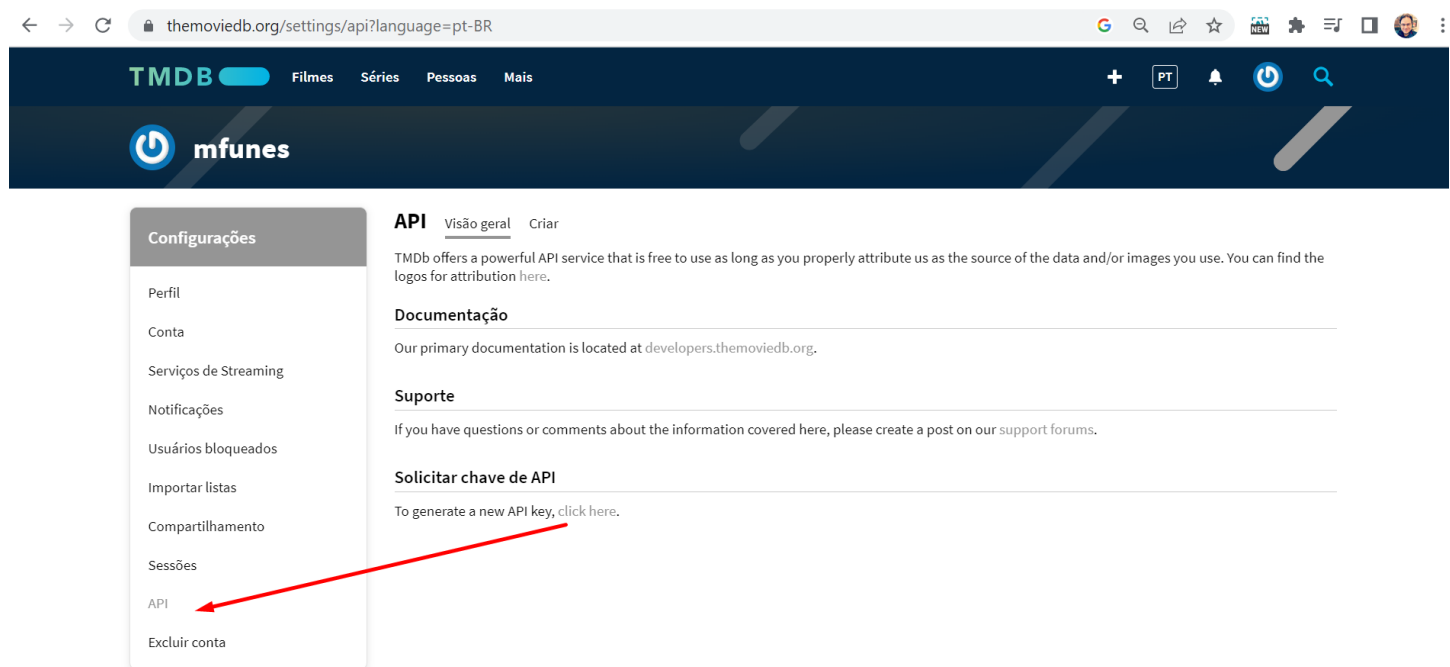
20 – Para obter as informações de filmes iremos usar a API do TMDb, acesse o site <https://www.themoviedb.org/?language=pt-BR> e crie uma conta.



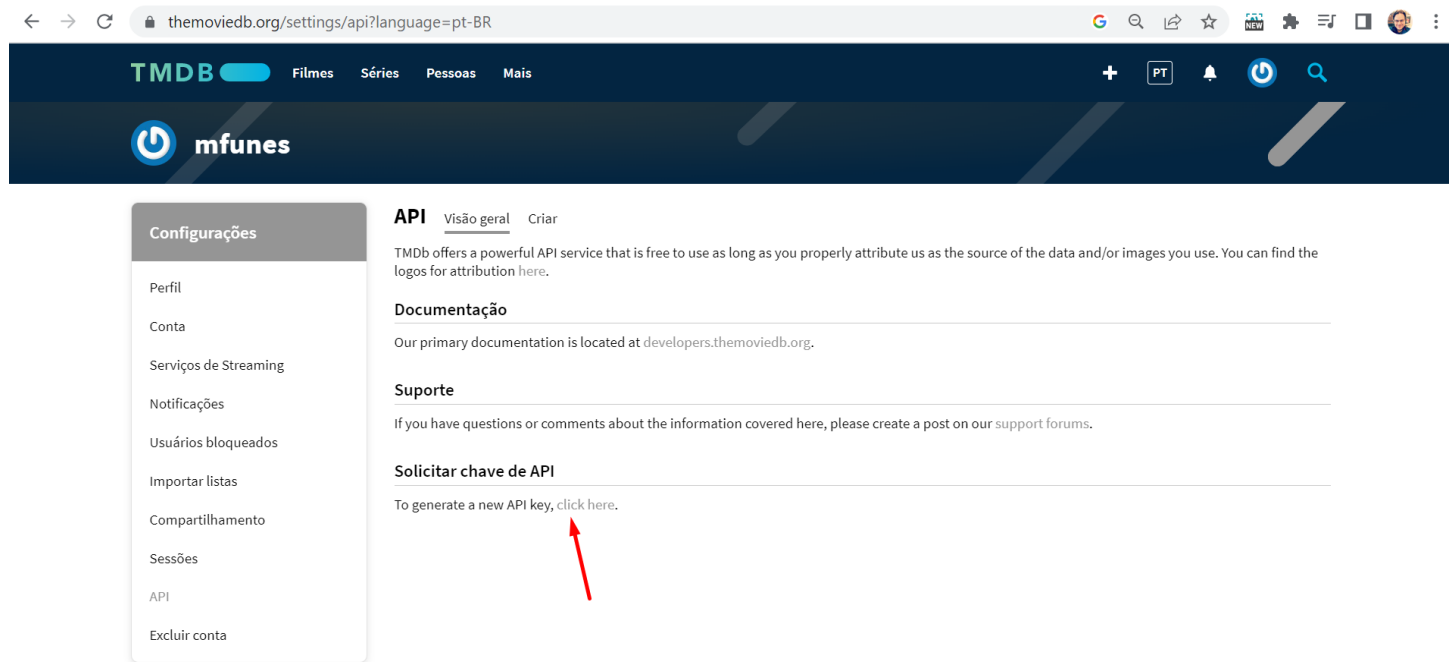
21 – Após criar a conta e logar (ele envia um e-mail de autenticação). Vá e, Configurações



22 – Depois vá em API



23 – Clique em Solicitar API



The screenshot shows the TMDb API settings page in Portuguese. The left sidebar contains a 'Configurações' menu with options like Perfil, Conta, and API. The main content area has tabs for 'API', 'Visão geral', and 'Criar'. Under the 'API' tab, there are sections for 'Documentação', 'Suporte', and 'Solicitar chave de API'. A red arrow points to the text 'To generate a new API key, click here.' in the 'Solicitar chave de API' section.

themoviedb.org/settings/api?language=pt-BR

TMDb Filmes Séries Pessoas Mais

mfunes

Configurações

- Perfil
- Conta
- Serviços de Streaming
- Notificações
- Usuários bloqueados
- Importar listas
- Compartilhamento
- Sessões
- API
- Excluir conta

API Visão geral Criar

TMDb offers a powerful API service that is free to use as long as you properly attribute us as the source of the data and/or images you use. You can find the logos for attribution [here](#).

Documentação

Our primary documentation is located at [developers.themoviedb.org](#).

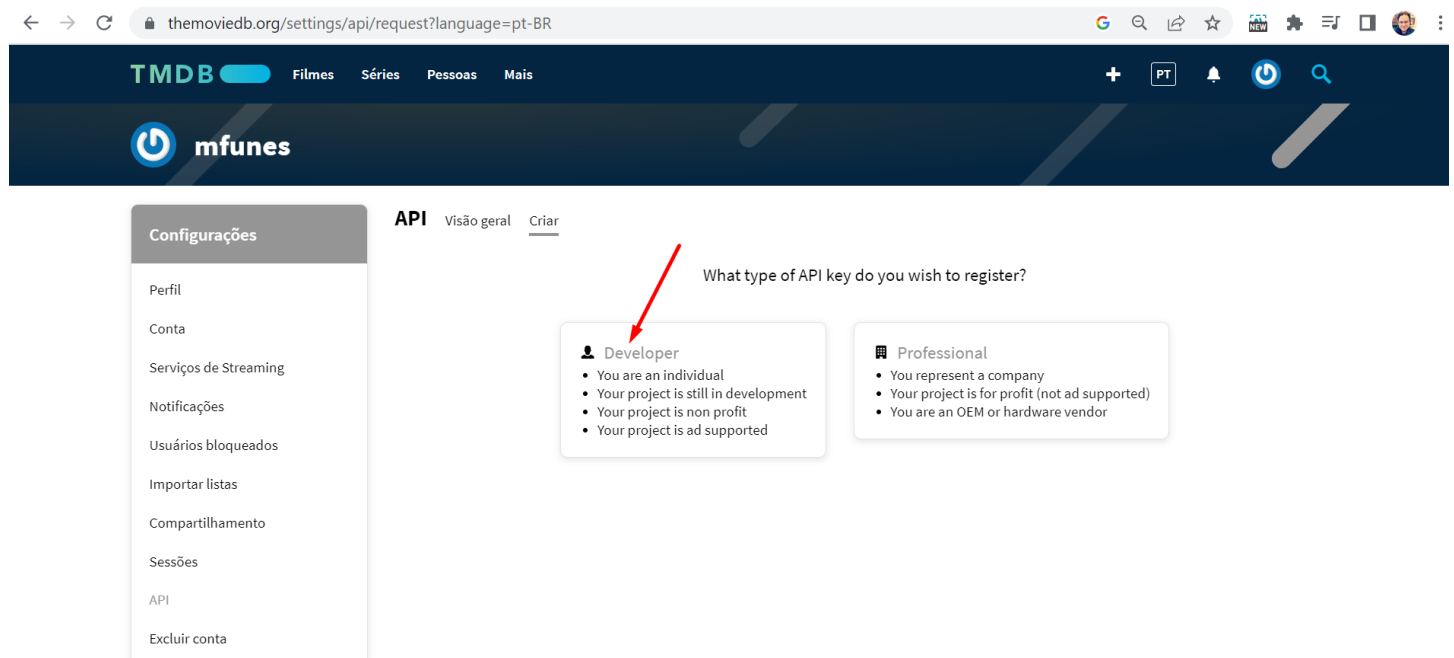
Suporte

If you have questions or comments about the information covered here, please create a post on our support forums.

Solicitar chave de API

To generate a new API key, [click here](#).

24 – Clique em Developer



The screenshot shows the TMDb API request page in Portuguese. The left sidebar is the same as in the previous image. The main content area has tabs for 'API', 'Visão geral', and 'Criar'. Under the 'API' tab, there is a heading 'What type of API key do you wish to register?' followed by two options: 'Developer' and 'Professional'. A red arrow points to the 'Developer' option.

themoviedb.org/settings/api/request?language=pt-BR

TMDb Filmes Séries Pessoas Mais

mfunes

Configurações

- Perfil
- Conta
- Serviços de Streaming
- Notificações
- Usuários bloqueados
- Importar listas
- Compartilhamento
- Sessões
- API
- Excluir conta

API Visão geral Criar

What type of API key do you wish to register?

Developer

- You are an individual
- Your project is still in development
- Your project is non profit
- Your project is ad supported

Professional

- You represent a company
- Your project is for profit (not ad supported)
- You are an OEM or hardware vendor

25 – Aceite os termos no final da página.

← → ↻ themoviedb.org/settings/api/new?type=developer

obligation for or on behalf of TMDB, express or implied, and you shall not attempt to bind TMDB to any contract.

2. **Invalidity of Specific Terms.** If any provision of the Terms of Use is found by a court of competent jurisdiction to be invalid, the parties nevertheless agree that the other provisions of this agreement will remain in full force and effect and the court should endeavor to give effect to the parties' intentions as reflected in the invalid provision.

3. **Location of Lawsuit and Choice of Law.** THE TERMS OF USE AND THE RELATIONSHIP BETWEEN YOU AND TMDB SHALL BE GOVERNED BY THE LAWS OF THE STATE OF CALIFORNIA WITHOUT REGARD TO ITS CONFLICT OF LAW PROVISIONS. YOU AND TMDB AGREE TO SUBMIT TO THE PERSONAL JURISDICTION OF THE COURTS LOCATED WITHIN THE COUNTY OF SAN MATEO, CALIFORNIA.

4. **No Waiver of Rights by TMDB.** TMDB's failure to exercise or enforce any right or provision of the Terms of Use shall not constitute a waiver of such right or provision.

5. **No Transfer.** The rights and obligations of these Terms of Use is personal to you and may not be transferred by you, either voluntarily or by operation of law.

6. **Notice.** Any notice to be sent to you under these Terms of Use may be sent via email, post, or any other reasonable means, at the contact information provided by you to TMDB from time to time. It is your obligation to insure that this information is current.

Miscellaneous. The section headings and subheadings contained in this agreement are included for convenience only, and shall not limit or otherwise affect the terms of the Terms of Use. Any construction or interpretation to be made of the Terms of Use shall not be construed against the drafter. The Terms of Use constitute the entire agreement between TMDB and you with respect to the subject matter hereof.

This Agreement was last updated on: July 28, 2014.

Cancel Aceitar

THE MOVIE DB

Olá, mfunes!

O BÁSICO

- Sobre o TMDB
- Contate-nos
- Suporte
- API
- Status do Sistema

ENVOLVA-SE

- Bíblia do Colaborador
- Adicionar um novo Filme
- Adicionar uma nova Série

COMUNIDADE

- Diretrizes
- Discussões
- Placar de colaboradores
- Twitter

LEGALIDADE

- Termos de uso
- Termos de Uso da API
- Política de privacidade

26 – Coloque algumas informações sobre o motivo de usarmos a API (isso não influencia em nada ok?), depois clique em enviar.

TMDB Filmes Séries Pessoas Mais

mfunes

Configurações

- Perfil
- Conta
- Serviços de Streaming
- Notificações
- Usuários bloqueados
- Importar listas
- Compartilhamento
- Sessões
- API
- Excluir conta

API Visão geral Criar

Tipo de uso
Formação acadêmica ▼

Nome do aplicativo
Prime Flix

URL do aplicativo

Sobre o aplicativo
Projeto para aprendizado de React e API

Primeiro nome

Último nome

Endereço de e-mail
marciofun.es@gmail.com

Número de telefone (no parenthesis or dashes)

Endereço 1

Endereço 2

Cidade

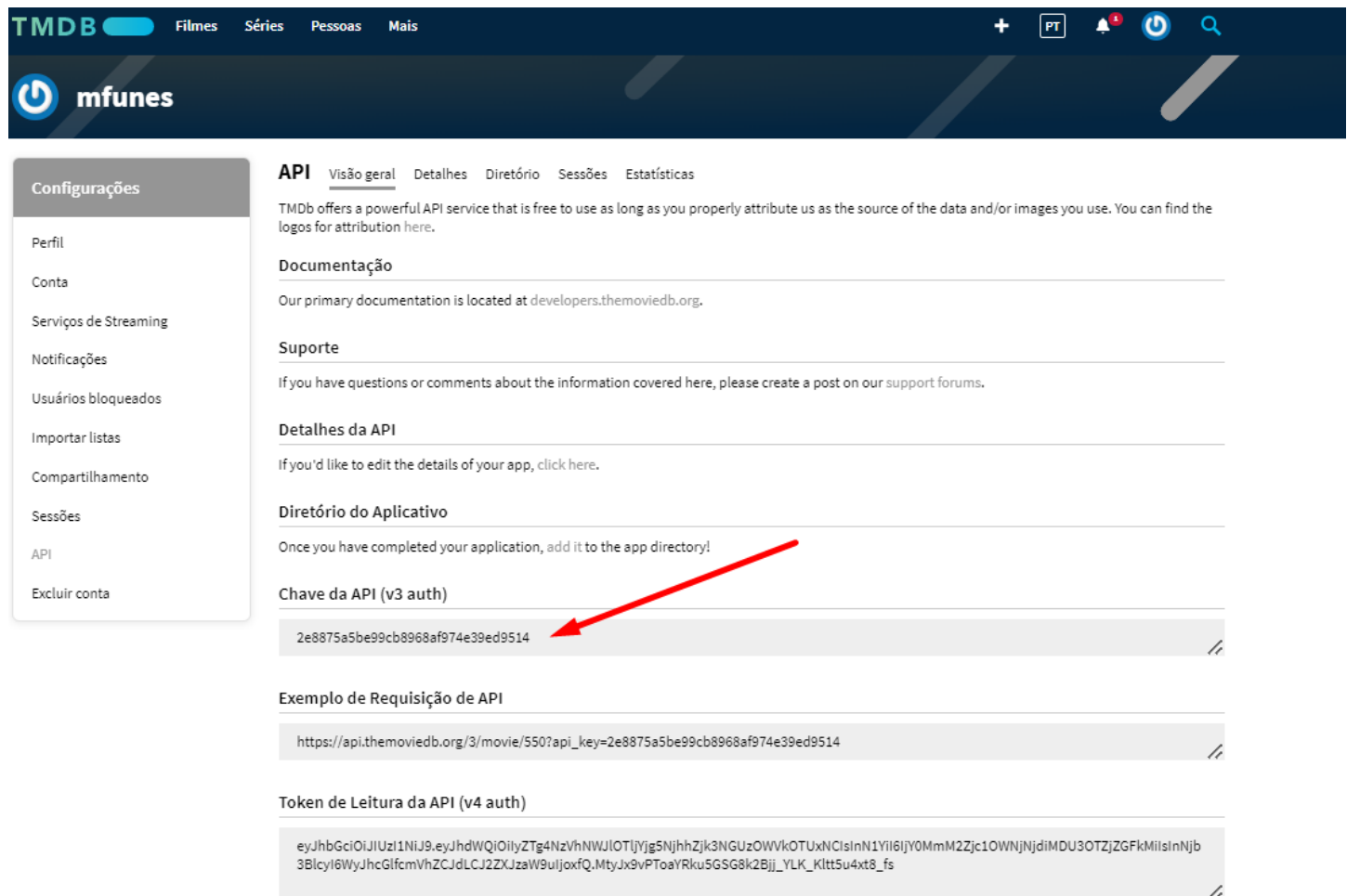
Estado

CEP

País
Brazil ▼

Enviar

27 – Agora temos uma chave API, isso quer dizer que toda vez que requisitamos um dado para este serviço, precisamos passar nossa chave, isso permite a aplicação confiar em nossa aplicação que está requisitando um dado do banco de dados do TMDb e de fato enviar o dado que queremos.



The screenshot shows the TMDb API configuration page within the mfun.es application. The page has a dark blue header with the TMDb logo and navigation links for Filmes, Séries, Pessoas, and Mais. Below the header is a sidebar with the title 'Configurações' and a list of settings: Perfil, Conta, Serviços de Streaming, Notificações, Usuários bloqueados, Importar listas, Compartilhamento, Sessões, API, and Excluir conta. The main content area is titled 'API' and includes tabs for Visão geral, Detalhes, Diretório, Sessões, and Estatísticas. The 'Visão geral' tab is active, displaying information about the TMDb API service, documentation, support, and details about the API. A red arrow points to the 'Chave da API (v3 auth)' field, which contains the value '2e8875a5be99cb8968af974e39ed9514'. Below this, there is an 'Exemplo de Requisição de API' field showing a URL with the API key as a query parameter. At the bottom, there is a 'Token de Leitura da API (v4 auth)' field containing a long alphanumeric string.

Configurações

- Perfil
- Conta
- Serviços de Streaming
- Notificações
- Usuários bloqueados
- Importar listas
- Compartilhamento
- Sessões
- API
- Excluir conta

API Visão geral Detalhes Diretório Sessões Estatísticas

TMDb offers a powerful API service that is free to use as long as you properly attribute us as the source of the data and/or images you use. You can find the logos for attribution [here](#).

Documentação

Our primary documentation is located at [developers.themoviedb.org](#).

Suporte

If you have questions or comments about the information covered here, please create a post on our support forums.

Detalhes da API

If you'd like to edit the details of your app, [click here](#).

Diretório do Aplicativo

Once you have completed your application, add it to the app directory!

Chave da API (v3 auth)

2e8875a5be99cb8968af974e39ed9514

Exemplo de Requisição de API

https://api.themoviedb.org/3/movie/550?api_key=2e8875a5be99cb8968af974e39ed9514

Token de Leitura da API (v4 auth)

eyJhbGciOiJIUzI1NiJ9.eyJhdWQiOiIyZTg4NzVhNWJlOTIjYjg5NjhhZjk3NGUzOWVkb0UxNCIsbnN1YiI6IjY0MmM2Zjc1OWNjNjdiMDU3OTZjZGFkMlIsbnNjb3BlcyI6WjYhcGlfcmlVhZCJdLCJ2ZXJzaW9uIjoxfQ.MtyJx9vPToaYRku5GSG8k2Bjj_YLK_KlItt5u4xt8_fs

28 – Copie este exemplo de Requisição de API e cole em uma nova aba do navegador.

TMDB

FilmesSériesPessoasMais

+PT🔔⚙️🔍

mfunes

Configurações

Perfil

Conta

Serviços de Streaming

Notificações

Usuários bloqueados

Importar listas

Compartilhamento

Sessões

API

Excluir conta

API

Visão geralDetalhesDiretórioSessõesEstatísticas

TMDb offers a powerful API service that is free to use as long as you properly attribute us as the source of the data and/or images you use. You can find the logos for attribution here.

Documentação

Our primary documentation is located at developers.themoviedb.org.

Suporte

If you have questions or comments about the information covered here, please create a post on our support forums.

Detalhes da API

If you'd like to edit the details of your app, click here.

Diretório do Aplicativo

Once you have completed your application, add it to the app directory!

Chave da API (v3 auth)

2e8875a5be99cb8968af974e39ed9514

Exemplo de Requisição de API

`https://api.themoviedb.org/3/movie/550?api_key=2e8875a5be99cb8968af974e39ed9514`

Token de Leitura da API (v4 auth)

`eyJhbGciOiJIUzI1NiIsInR5cGU6ImFwa2VkaWkiLCJpdjYg5NjhZjk3NGUzOWVkbOTUxNCIsbnN1YiI6ImMmM2Zjc1OWNjNDIMDU3OTZjZGFkMiIsbnNjb3BlcyI6WyJhcGlfcmlhbm9uZCJdLCJzaW9uIjoxfQ.MtyJx9vPToaYRku5GSg8k2Bjj_YLK_Kltt5u4xt8_fs`

29 – Entender como a requisição funciona é o passo mais importante, caso contrário não conseguimos programar corretamente a requisição na nossa aplicação. Veja que no exemplo temos uma URL base: `api.themoviedb.org/3/movie/` e para buscar filmes diferentes passamos o id do filme, nesse exemplo o id 550 é referente ao filme “Fight Club”, quando você pressionou enter nesse exemplo ele retornou um JSON com as informações do filme, será a mesma informação que a nossa aplicação também irá receber quando requisitar o mesmo endereço. Veja também que na mesma URL tivemos que passar o parâmetro `api_key` que contém nossa chave única, na URL de uma API tudo que vem depois de `?` Será parâmetros de configuração.

```
← → ↻ https://api.themoviedb.org/3/movie/550?api_key=2e8875a5be99cb8968af974e39ed9514

1 // 20230404155718
2 // https://api.themoviedb.org/3/movie/550?api_key=2e8875a5be99cb8968af974e39ed9514
3
4 {
5   "adult": false,
6   "backdrop_path": "/hZkgoQYus5vegHoetLkCJzb17z1.jpg",
7   "belongs_to_collection": null,
8   "budget": 63000000,
9   "genres": [
10    {
11      "id": 18,
12      "name": "Drama"
13    },
14    {
15      "id": 53,
16      "name": "Thriller"
17    },
18    {
19      "id": 35,
20      "name": "Comedy"
21    }
22  ],
23   "homepage": "http://www.foxmovies.com/movies/fight-club",
24   "id": 550,
25   "imdb_id": "tt0137523",
26   "original_language": "en",
27   "original_title": "Fight Club",
28   "overview": "A ticking-time-bomb insomniac and a slippery soap salesman channel primal male aggression into a shocking new form of therapy. Their
forming in every town, until an eccentric gets in the way and ignites an out-of-control spiral toward oblivion.",
29   "popularity": 75.78,
30   "poster_path": "/pB8BM7pdSp6B6Ih7QZ4DrQ3Pm7K.jpg",
31   "production_companies": [
32     {
33       "id": 508,
34       "logo_path": "/7cxRWzi4LsVm4Utfpr1hfARNurT.png",
35       "name": "Regency Enterprises",
```

30 – Observe que no site temos o link da documentação de como essa API funciona, sem a documentação da API não conseguimos usar corretamente. Abra a documentação.

TMDB

Filmes

Séries

Pessoas

Mais

+

PT

Configurações

Perfil

Conta

Serviços de Streaming

Notificações

Usuários bloqueados

Importar listas

Compartilhamento

Sessões

API

Excluir conta

API

Visão geral

Detalhes

Diretório

Sessões

Estatísticas

TMDB offers a powerful API service that is free to use as long as you properly attribute us as the source of the data and/or images you use. You can find the logos for attribution [here](#).

Documentação

Our primary documentation is located at developers.themoviedb.org.

Suporte

If you have questions or comments about the information covered here, please create a post on our [support forums](#).

Detalhes da API

If you'd like to edit the details of your app, [click here](#).

Diretório do Aplicativo

Once you have completed your application, [add it to the app directory!](#)

Chave da API (v3 auth)

2e8875a5be99cb8968af974e39ed9514

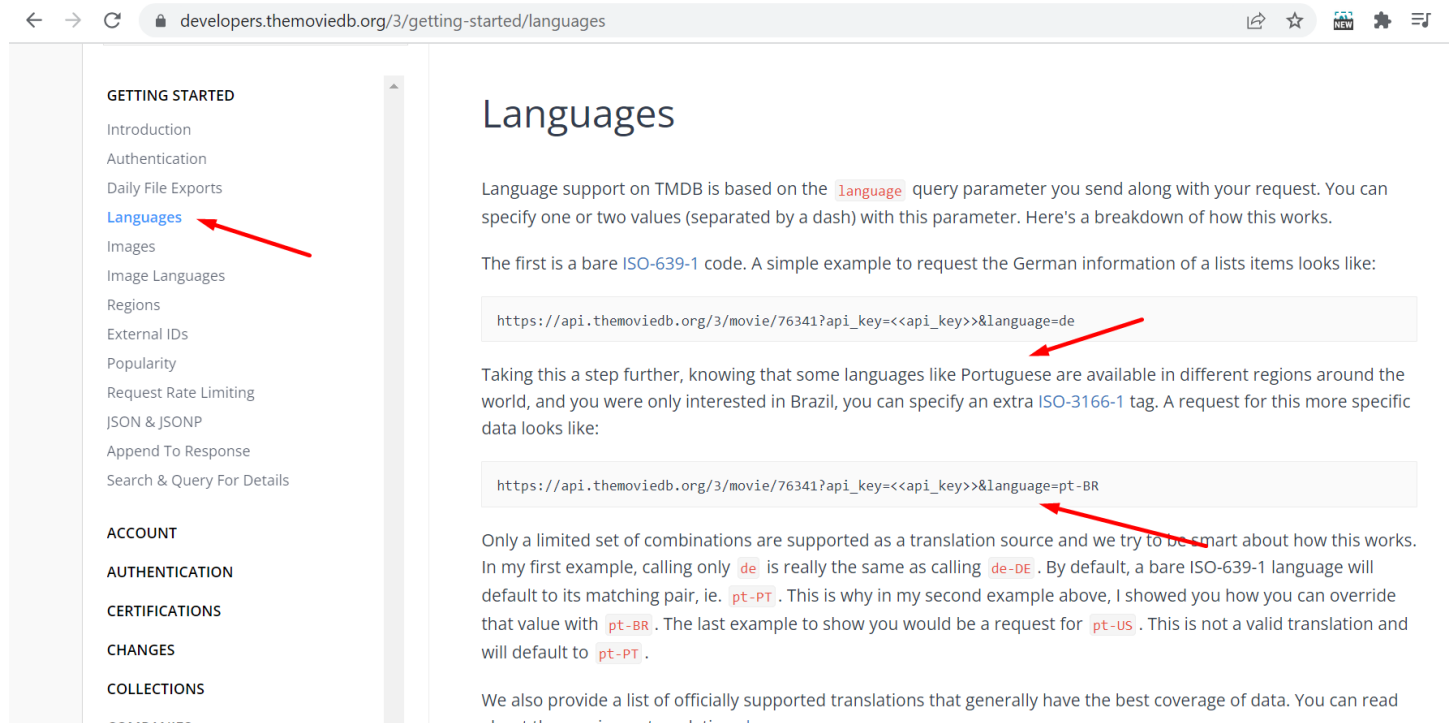
Exemplo de Requisição de API

`https://api.themoviedb.org/3/movie/550?api_key=2e8875a5be99cb8968af974e39ed9514`

Token de Leitura da API (v4 auth)

`eyJhbGciOiJIUzI1NiJ9.eyJhdWQiOiIyZTg4NmZhNWJlOTIyYjg5NjhhZjZk3NGUzOWVkbOTUxNCIsInN1YiI6IjY0MmM2Zjc1OWNjNjdiMDU3OTZjZGFkMiIsInNjb3BlcyI6WyJhcGlcmVhZCJdLCJ2ZXJzaW9uIjoxfQ.MtyJx9vPToaYRku5SGS8k2Bjj_YLK_Kltt5u4xt8_fs`

31 – Veja que na sessão Language da documentação podemos usar um novo parâmetro chamado language onde permite mudarmos o idioma das informações consultadas.



The screenshot shows the TMDb API documentation page for Languages. The left sidebar contains a navigation menu with sections: GETTING STARTED, ACCOUNT, AUTHENTICATION, CERTIFICATIONS, CHANGES, and COLLECTIONS. Under GETTING STARTED, the 'Languages' link is highlighted in blue and pointed to by a red arrow. The main content area is titled 'Languages' and explains that language support is based on the 'language' query parameter. It provides two examples of API requests: one for German (de) and one for Portuguese from Brazil (pt-BR). Both examples are highlighted in light gray boxes and pointed to by red arrows. The text explains that a bare ISO-639-1 code (like 'de') defaults to its matching pair (like 'de-DE'), and that more specific codes (like 'pt-BR') can be used to override the default. It also mentions that a request for 'pt-US' is not valid and will default to 'pt-PT'. At the bottom, it states that a list of officially supported translations is available, with a link to 'about these primary translations here'.

← → ↺ developers.themoviedb.org/3/getting-started/languages

GETTING STARTED

- Introduction
- Authentication
- Daily File Exports
- Languages**
- Images
- Image Languages
- Regions
- External IDs
- Popularity
- Request Rate Limiting
- JSON & JSONP
- Append To Response
- Search & Query For Details

ACCOUNT

AUTHENTICATION

CERTIFICATIONS

CHANGES

COLLECTIONS

Languages

Language support on TMDb is based on the `language` query parameter you send along with your request. You can specify one or two values (separated by a dash) with this parameter. Here's a breakdown of how this works.

The first is a bare [ISO-639-1](#) code. A simple example to request the German information of a lists items looks like:

```
https://api.themoviedb.org/3/movie/76341?api_key=<<api_key>>&language=de
```

Taking this a step further, knowing that some languages like Portuguese are available in different regions around the world, and you were only interested in Brazil, you can specify an extra [ISO-3166-1](#) tag. A request for this more specific data looks like:

```
https://api.themoviedb.org/3/movie/76341?api_key=<<api_key>>&language=pt-BR
```

Only a limited set of combinations are supported as a translation source and we try to be smart about how this works. In my first example, calling only `de` is really the same as calling `de-DE`. By default, a bare ISO-639-1 language will default to its matching pair, ie. `pt-PT`. This is why in my second example above, I showed you how you can override that value with `pt-BR`. The last example to show you would be a request for `pt-US`. This is not a valid translation and will default to `pt-PT`.

We also provide a list of officially supported translations that generally have the best coverage of data. You can read [about these primary translations here](#).

32 – Coloque o & pois toda vez que adicionamos um novo parâmetro a URL da API temos que concatenar, depois language=pt-br, veja que agora a API retorna o JSON na lingua portuguesa.

```
← → ↻ 🔒 api.themoviedb.org/3/movie/550?api_key=2e8875a5be99cb8968af974e39ed9514&language=pt-br

1 // 20230404160634
2 // https://api.themoviedb.org/3/movie/550?api_key=2e8875a5be99cb8968af974e39ed9514&language=pt-br
3
4 {
5   "adult": false,
6   "backdrop_path": "/h7kgoQYus5vegHoetLkC7zb17zJ.jpg",
7   "belongs_to_collection": null,
8   "budget": 63000000,
9   "genres": [
10    {
11      "id": 18,
12      "name": "Drama"
13    },
14    {
15      "id": 53,
16      "name": "Thriller"
17    },
18    {
19      "id": 35,
20      "name": "Comédia"
21    }
22  ],
23   "homepage": "",
24   "id": 550,
25   "imdb_id": "tt0137523",
26   "original_language": "en",
27   "original_title": "Fight Club",
28   "overview": "Um homem deprimido que sofre de insônia conhece um estranho vendedor de sabonetes chamado Tyler Durden. Eles formam um clube clandestino com reg  
cansados de suas vidas mundanas. Mas sua parceria perfeita é comprometida quando Marla chama a atenção de Tyler.",
29   "popularity": 75.78,
30   "poster_path": "/r3pPehX4ik8NLYPpbDRAh0YRtMb.jpg",
31   "production_companies": [
32     {
33       "id": 508,
34       "logo_path": "/7cxRWzi4LsVm4Utfr1hfARNurT.png",
35       "name": "Regency Enterprises",
```

33 – Volte na documentação e vá na sessão Movies, veja que existem vários endpoints diferentes, cada um retorna diferentes informações.

← → ↻ developers.themoviedb.org/3/movies/get-movie-details

MOVIES

- GET [Get Details](#)
- GET Get Account States
- GET Get Alternative Titles
- GET Get Changes
- GET Get Credits
- GET Get External IDs
- GET Get Images
- GET Get Keywords
- GET Get Lists
- GET Get Recommendations
- GET Get Release Dates
- GET Get Reviews
- GET Get Similar Movies
- GET Get Translations
- GET Get Videos
- GET Get Watch Providers
- POST Rate Movie
- DELETE Delete Rating
- GET Get Latest
- GET Get Now Playing
- GET Get Popular
- GET Get Top Rated

Path Parameters

movie_id	integer	required
----------	---------	----------

Query String

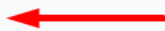
api_key	string	default: <<api_key>>	required
language	string	Pass a ISO 639-1 value to display translated data for the fields that support it. minLength: 2 pattern: ([a-z]{2})-([A-Z]{2}) default: en-US	optional
append_to_response	string	Append requests within the same namespace to the response. pattern: ([\w]+)	optional

Responses application/json

● 200	Schema Example collapse all												
● 401	object												
● 404													
	<table><tr><td>adult</td><td>boolean</td><td>optional</td></tr><tr><td>backdrop_path</td><td>string or null</td><td>optional</td></tr><tr><td>belongs_to_collection</td><td>null or object</td><td>optional</td></tr><tr><td>budget</td><td>integer</td><td>optional</td></tr></table>	adult	boolean	optional	backdrop_path	string or null	optional	belongs_to_collection	null or object	optional	budget	integer	optional
adult	boolean	optional											
backdrop_path	string or null	optional											
belongs_to_collection	null or object	optional											
budget	integer	optional											

34 – Vá na opção `now_playing` na lista de endpoints dentro de Movies, veja que podemos buscar somente os filmes em cartaz, para isso precisamos entender como a Query deve funcionar. Nesse exemplo temos que passar a `"/now_playing"` além disso podemos colocar a região do mundo, também mudar a linguagem, isso permitiria ver qual filme está em cartaz nos EUA ou na Europa.

Get Now Playing

GET `/movie/now_playing` 

Get a list of movies in theatres. This is a release type query that looks for all movies that have a release type of 2 or 3 within the specified date range.

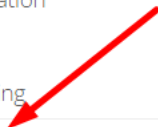
You can optionally specify a `region` parameter which will narrow the search to only look for theatrical release dates within the specified country.

Definition Try it out

Authentication

☒ API Key

Query String



<code>api_key</code>	string	default: <<api_key>>	required
<code>language</code>	string	Pass a ISO 639-1 value to display translated data for the fields that support it. minLength: 2 pattern: ([a-z]{2})-([A-Z]{2}) default: en-US	optional
<code>page</code>	integer	Specify which page to query. minimum: 1 maximum: 1000 default: 1	optional
<code>region</code>	string	Specify a ISO 3166-1 code to filter release dates. Must be uppercase. pattern: ^[A-Z]{2}\$	optional

Responses

application/json



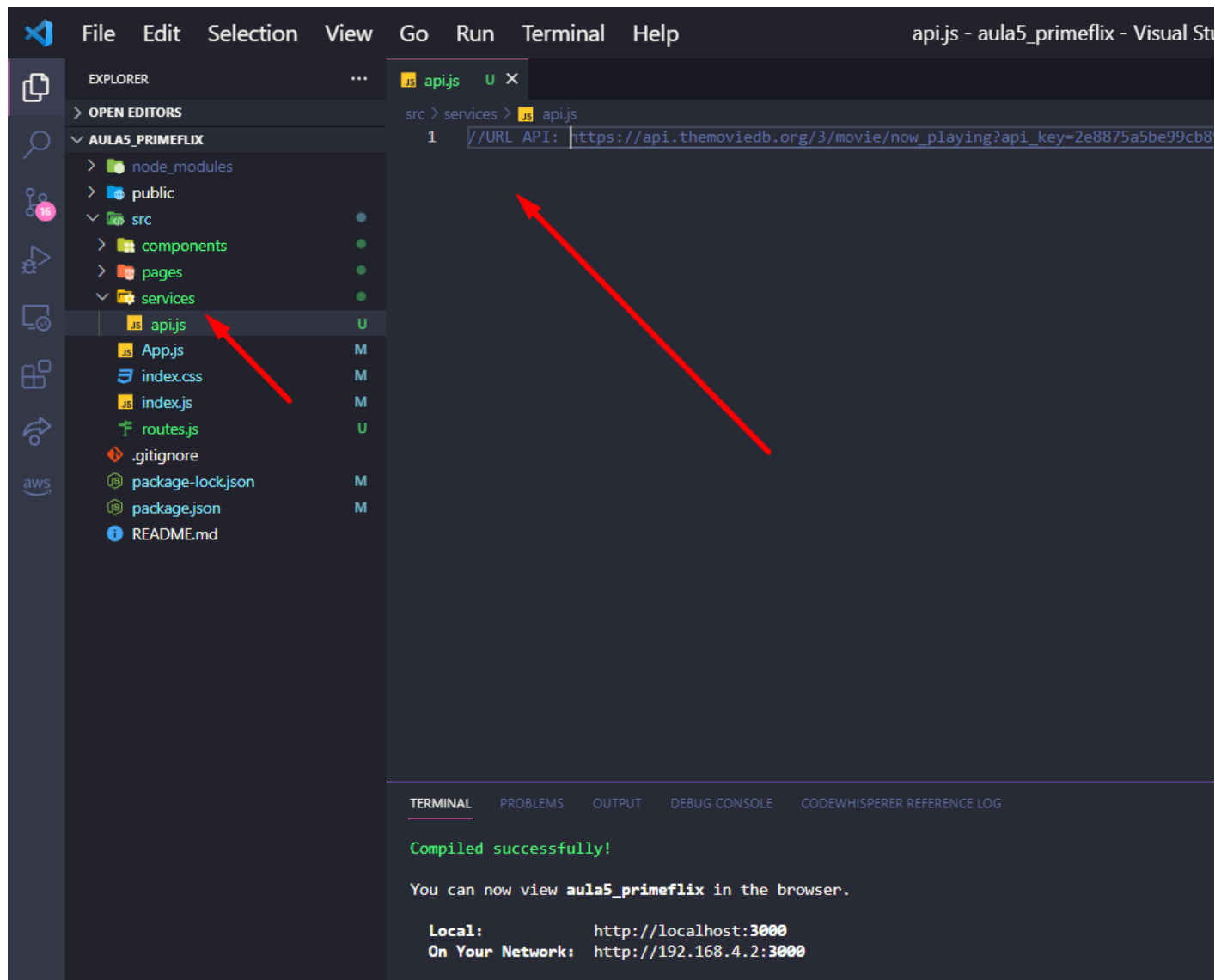
200	Schema Example																						
401	object																						
404	<table><tr><td><code>page</code></td><td>integer</td><td>optional</td></tr><tr><td><code>results</code></td><td>array[object] (Movie List Result Object)</td><td>optional</td></tr><tr><td><code>poster_path</code></td><td>string or null</td><td>optional</td></tr><tr><td><code>adult</code></td><td>boolean</td><td>optional</td></tr><tr><td><code>overview</code></td><td>string</td><td>optional</td></tr><tr><td><code>release_date</code></td><td>string</td><td>optional</td></tr><tr><td><code>genre_ids</code></td><td>array[integer]</td><td>optional</td></tr></table>	<code>page</code>	integer	optional	<code>results</code>	array[object] (Movie List Result Object)	optional	<code>poster_path</code>	string or null	optional	<code>adult</code>	boolean	optional	<code>overview</code>	string	optional	<code>release_date</code>	string	optional	<code>genre_ids</code>	array[integer]	optional	
<code>page</code>	integer	optional																					
<code>results</code>	array[object] (Movie List Result Object)	optional																					
<code>poster_path</code>	string or null	optional																					
<code>adult</code>	boolean	optional																					
<code>overview</code>	string	optional																					
<code>release_date</code>	string	optional																					
<code>genre_ids</code>	array[integer]	optional																					

35 – Testando isso no navegador mude para *movie/now_playing* mantendo o resto da URL igual. Ele retonar os filmes em cartaz

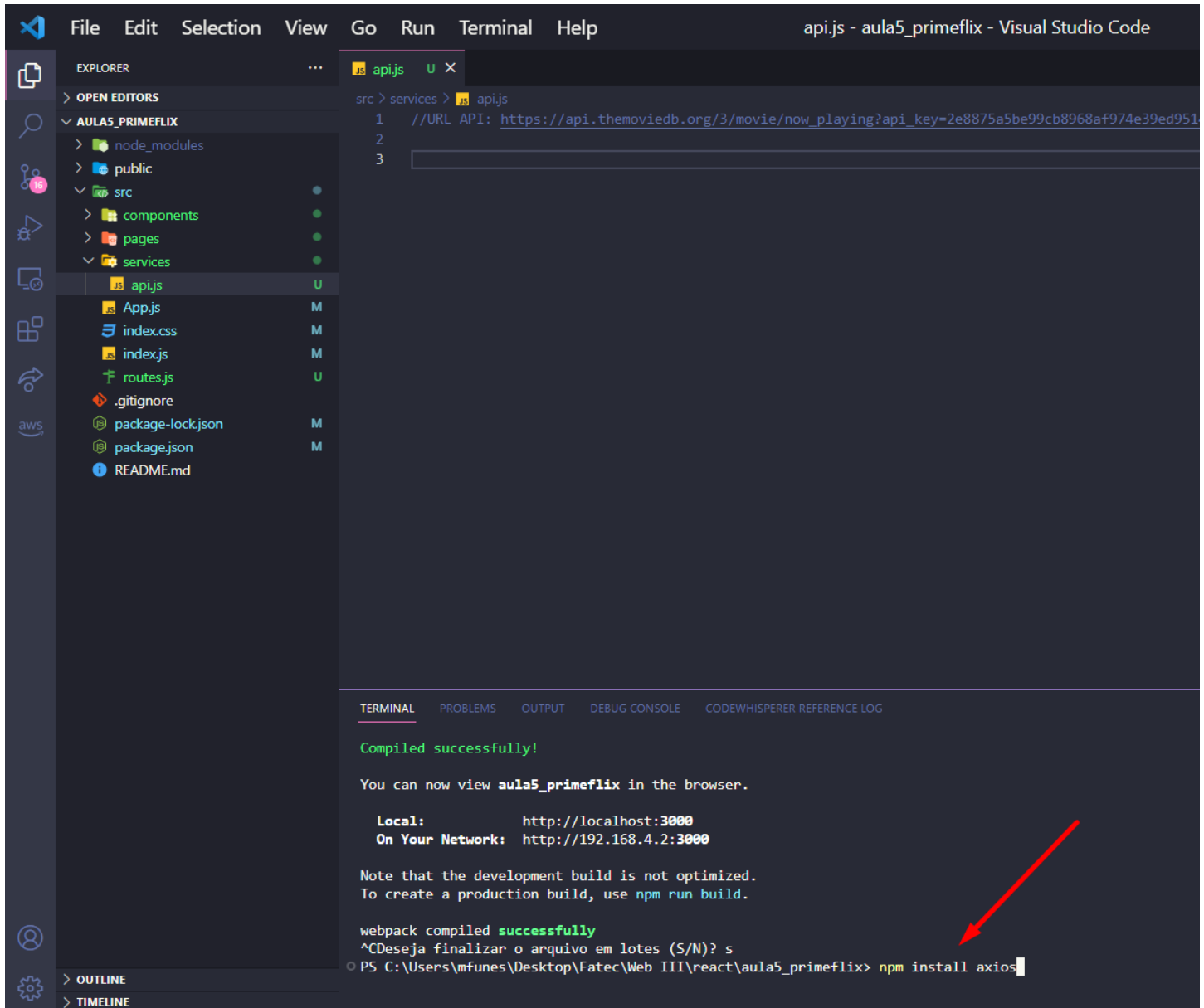
```
← → ↻ api.themoviedb.org/3/movie/now_playing?api_key=2e8875a5be99cb8968af974e39ed9514&language=pt-br 🔍 📌 ☆ 🌐
```

```
74 ],
75 "id": 804150,
76 "original_language": "en",
77 "original_title": "Cocaine Bear",
78 "overview": "Um grupo excêntrico de policiais, criminosos, turistas e adolescentes se convergem numa floresta da Geórgia, onde um urso negro de 200kg ingeriu uma quantidade in
cocaina e partiu para uma fúria movida a coca por mais golpe... e sangue.",
79 "popularity": 2673.172,
80 "poster_path": "/kCULfhcKxjzYD2NweXk8dEzeLFC.jpg",
81 "release_date": "2023-02-22",
82 "title": "O Urso do Pó Branco",
83 "video": false,
84 "vote_average": 6.5,
85 "vote_count": 622
86 },
87 {
88 "adult": false,
89 "backdrop_path": "/i8dshlvq4LE3s0v8PrkDdUyb1ae.jpg",
90 "genre_ids": [
91 28,
92 53,
93 80
94 ],
95 "id": 603692,
96 "original_language": "en",
97 "original_title": "John Wick: Chapter 4",
98 "overview": "Com o preço por sua cabeça cada vez maior, John Wick leva sua luta contra a alta mesa global enquanto procura os jogadores mais poderosos do submundo, de Nova Yor
Osaka a Berlim.",
99 "popularity": 2600.628,
100 "poster_path": "/97NPC9AI2EptBFj1MRN9psnvT13.jpg",
101 "release_date": "2023-03-22",
102 "title": "John Wick 4: Baba Yaga",
103 "video": false,
104 "vote_average": 8.1,
105 "vote_count": 710
106 }
```

36 – Agora que entendemos como a API funciona, vamos criar um serviço que irá consumir a API e mostrar as informações na nossa aplicação. Dentro de SRC crie uma pasta chamada services e dentro de services crie um arquivo chamado api.js. Cole o endereço da API que usamos no navegador de modo comentado // só para não esquecer como requisitar corretamente.



37 – Para as requisições de API vamos utilizar o Axios que é um cliente para requisições HTTP muito famoso do Node. Precisamos instalar ele em nossa aplicação, pare a execução da aplicação (CTRL + C) e digite **npm install axios**. Após a instalação rode o projeto novamente com **npm start**.



The screenshot shows the Visual Studio Code interface with the following components:

- EXPLORER:** Displays the project structure for 'aula5_primeflix'. The 'src' directory is expanded, showing 'components', 'pages', 'services', and 'apijs' (selected).
- EDITOR:** Shows the 'apijs' file with the following content:

```
src > services > apijs
1 //URL API: https://api.themoviedb.org/3/movie/now_playing?api_key=2e8875a5be99cb8968af974e39ed951
2
3
```
- TERMINAL:** Shows the output of the previous build and the command to install axios. A red arrow points to the command line.

```
Compiled successfully!

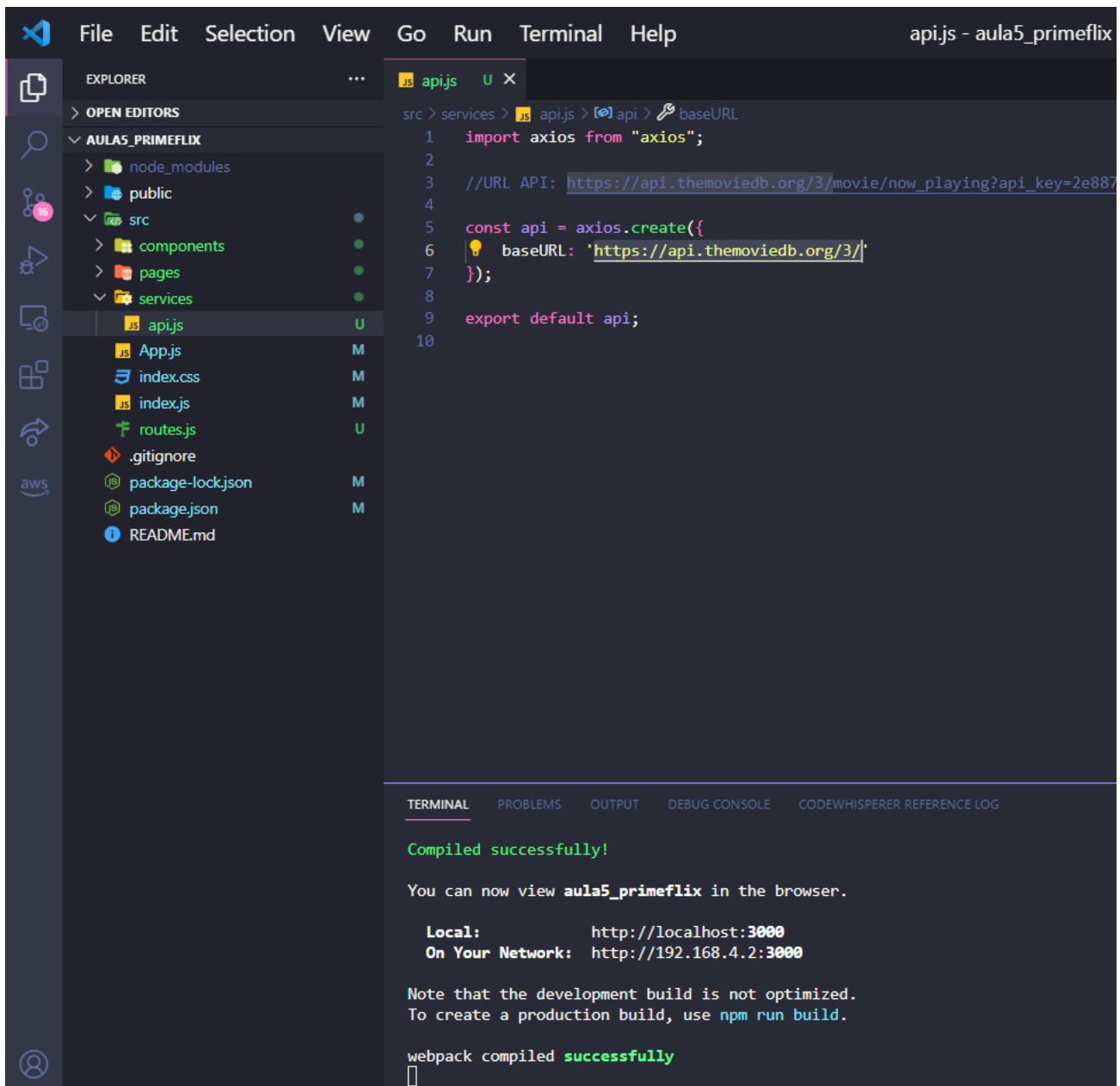
You can now view aula5_primeflix in the browser.

Local:      http://localhost:3000
On Your Network: http://192.168.4.2:3000

Note that the development build is not optimized.
To create a production build, use npm run build.

webpack compiled successfully
^CDesaja finalizar o arquivo em lotes (S/N)? s
PS C:\Users\mfunes\Desktop\Fatec\Web III\react\aula5_primeflix> npm install axios
```

38 – Faça a importação do axios na primeira linha do arquivo api.js, depois vamos criar uma variável constante (const api), ela terá como valor a base URL, isso facilita bastante nas requisições pois a URL base, <https://api.themoviedb.org/3/> nunca muda, segundo a documentação.



The screenshot shows the Visual Studio Code editor interface. On the left, the Explorer sidebar displays the project structure for 'aula5_primeflix', including folders like 'node_modules', 'public', 'src', 'components', 'pages', and 'services'. The 'api.js' file is selected under the 'services' folder. The main editor window shows the content of 'api.js' with the following code:

```
src > services > api.js > api > baseUrl
1  import axios from "axios";
2
3  //URL API: https://api.themoviedb.org/3/movie/now_playing?api_key=2e887
4
5  const api = axios.create({
6    baseUrl: 'https://api.themoviedb.org/3/'
7  });
8
9  export default api;
10
```

At the bottom of the editor, the Terminal panel is active, displaying the following output:

```
TERMINAL  PROBLEMS  OUTPUT  DEBUG CONSOLE  CODEWHISPERER REFERENCE LOG

Compiled successfully!

You can now view aula5_primeflix in the browser.

Local:      http://localhost:3000
On Your Network:  http://192.168.4.2:3000

Note that the development build is not optimized.
To create a production build, use npm run build.

webpack compiled successfully
```

38 – Agora que temos a URL base criada, vamos consumir a API dentro da nossa Home, vá até a pasta Home e abra o index.js. Dentro do index.js, faça a importação do useEffect e useState par nos auxiliar no consumo da API, crie um useState chamado filmes e setFilmes e dentro de useEffect vamos criar uma nova função assíncrona chamada loadFilmes.

Informação: funções assíncronas são aquelas que acessam ou buscam algum tipo de recurso em um dispositivo externo, como por exemplo um banco de dados, nesse tipo de função precisamos que nosso código espere que a resposta esteja disponível antes de executar a ação seguinte.

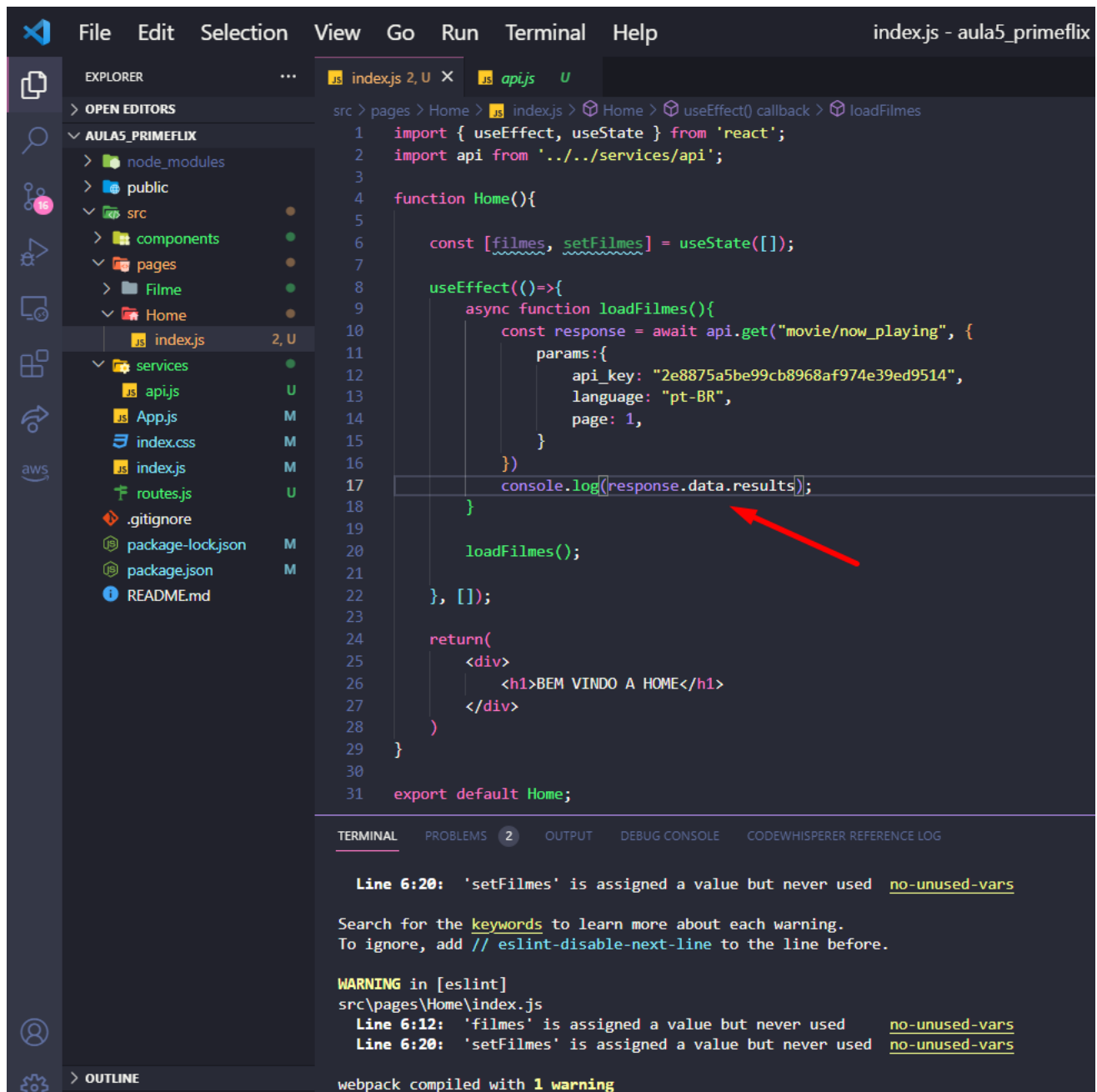
```
src > pages > Home >  index.js >  Home >  useEffect() callback
1  import { useEffect, useState } from 'react';
2  import api from '../services/api';
3
4  function Home(){
5
6      const [filmes, setFilmes] = useState([]);
7
8      useEffect(()=>{
9          async function loadFilmes(){
10              const response = await api.get("movie/now_playing", {
11                  params:{
12                      api_key: "2e8875a5be99cb8968af974e39ed9514",
13                      language: "pt-BR",
14                      page: 1,
15                  }
16              })
17              console.log(response);
18          }
19
20          loadFilmes();
21
22      }, []);
23
24      return(
25          <div>
26              <h1>BEM VINDO A HOME</h1>
27          </div>
28      )
29  }
30
31  export default Home;
```

39 - Perceba que criamos uma const chamada response, ela recebe a nossa variável api e faz então um GET (poderia ser um POST caso a API trabalhasse com POST), nesse GET passamos o complemento da URL base que é “movie/now_playing”, além disso podemos configurar os parametros com “params”, passando então a api_key (coloque aqui a sua API qui gerada no site do TMDb), além do language e page para retormar apenas a primeira página de dados da API. Coloque apenas um console.log para ver se está funcionando corretamente. Vá até o navegador, abra o console e veja o retorno da nossa requisição, ele retornou 19 filmes com essa requisição.

The screenshot shows a web browser at localhost:3000 displaying a movie application. The interface has a header with "Prime Flix" and a button "Meus Filmes". Below the header, it says "BEM VINDO A HOME". The browser's developer console is open, showing the network tab with a request to "react-dom.development.js:29840". The console log shows the response of the API call, which is a JSON object. The response is expanded, showing the "data" property, which is an array of 20 movie objects. The first 19 movies are listed, each with properties like "adult", "backdrop_path", "genre_ids", and "id". The "total_pages" property is 85, and the "total_results" property is 1682. Red arrows point to the "data" property, the "page" property (value 1), the "total_pages" property (value 85), and the "total_results" property (value 1682).

```
react-dom.development.js:29840
Download the React DevTools for a better development experience: https://reactjs.org/link/react-devtools
index.js:17
{data: {...}, status: 200, statusText: '', headers: AxiosHeaders, config: {...}, ...}
index.js:17
▼ {data: {...}, status: 200, statusText: '', headers: AxiosHeaders, config: {...}, ...}
  ▶ config: {transitional: {...}, adapter: Array(2), transformRequest: Array(1), transformResponse: Array(1), timeout: ...}
  ▼ data:
    ▶ dates: {maximum: '2023-04-02', minimum: '2023-02-13'}
    ▶ page: 1
    ▼ results: Array(20)
      ▶ 0: {adult: false, backdrop_path: '/wD2kUCX1Bb6oeIb2uz7kdbfLP6k.jpg', genre_ids: Array(2), id: 980078, origin ...}
      ▶ 1: {adult: false, backdrop_path: '/5i65jyObdWqyun8klucXr1fByw.jpg', genre_ids: Array(2), id: 677179, origin ...}
      ▶ 2: {adult: false, backdrop_path: '/m1fg6SLK0WvRpzM1AmZu38m0Tx8.jpg', genre_ids: Array(1), id: 842945, origin ...}
      ▶ 3: {adult: false, backdrop_path: '/a2tys4SD7xzVaogPntGst1ypVoT.jpg', genre_ids: Array(3), id: 804150, origin ...}
      ▶ 4: {adult: false, backdrop_path: '/i8dshLvq4LE3s0v8PrkDduyb1ae.jpg', genre_ids: Array(3), id: 603692, origin ...}
      ▶ 5: {adult: false, backdrop_path: '/vSU1s0b7dNHC7tJoExF1MBYmWYh.jpg', genre_ids: Array(5), id: 816904, origin ...}
      ▶ 6: {adult: false, backdrop_path: '/ouB7hwc1G7QI3INoYJHaZL4vOaa.jpg', genre_ids: Array(5), id: 315162, origin ...}
      ▶ 7: {adult: false, backdrop_path: '/zM9RGbJ8Z3UNpFOabcRqh0iVAYP.jpg', genre_ids: Array(4), id: 631842, origin ...}
      ▶ 8: {adult: false, backdrop_path: '/qai0MnVwPYCwOPSAAnHIHyZBVqnd.jpg', genre_ids: Array(4), id: 776835, origin ...}
      ▶ 9: {adult: false, backdrop_path: '/sRFx2XPjyL7nRKVRKXVG6D0bVQI.jpg', genre_ids: Array(2), id: 884184, origin ...}
      ▶ 10: {adult: false, backdrop_path: '/9Rq14Eyrf7Tu1xk0P17VcNbh1n.jpg', genre_ids: Array(3), id: 646389, origi ...}
      ▶ 11: {adult: false, backdrop_path: '/dlrWhn0G3AtxYUX2D9P2bmzcsvF.jpg', genre_ids: Array(3), id: 536554, origi ...}
      ▶ 12: {adult: false, backdrop_path: '/wybm5mviUXx1BmX44gtpow5Y9TB.jpg', genre_ids: Array(3), id: 594767, origi ...}
      ▶ 13: {adult: false, backdrop_path: '/gs1T8t964rYXyQcqrxFh77ikyb.jpg', genre_ids: Array(3), id: 640146, origi ...}
      ▶ 14: {adult: false, backdrop_path: '/pxJbfnMIQqxCrdeLD0zQnWr6ouL.jpg', genre_ids: Array(3), id: 1077280, origi ...}
      ▶ 15: {adult: false, backdrop_path: '/2E6pIblien789yWpaz8Yd2njzUI.jpg', genre_ids: Array(2), id: 1026563, origi ...}
      ▶ 16: {adult: false, backdrop_path: '/2rVkdZFLN6DI5Paorae9m4IRWN.jpg', genre_ids: Array(3), id: 493529, origi ...}
      ▶ 17: {adult: false, backdrop_path: '/zGoZB4CboMzY1z4G3nU6BwMD82.jpg', genre_ids: Array(3), id: 758009, origi ...}
      ▶ 18: {adult: false, backdrop_path: '/98Hak5vgStHnrQ90ZfDIHsto1hV.jpg', genre_ids: Array(3), id: 845659, origi ...}
      ▶ 19: {adult: false, backdrop_path: '/r7Dfg9aRZ78gJsmDLcirIILNH3d.jpg', genre_ids: Array(1), id: 785084, origi ...}
    length: 20
    ▶ [[Prototype]]: Array(0)
    total_pages: 85
    total_results: 1682
```

40 – Altere o console.log para pegarmos apenas o results. Assim focamos apenas na lista de filmes que a API retorna.



```
1 import { useEffect, useState } from 'react';
2 import api from '../services/api';
3
4 function Home(){
5
6   const [filmes, setFilmes] = useState([]);
7
8   useEffect(()=>{
9     async function loadFilmes(){
10       const response = await api.get("movie/now_playing", {
11         params:{
12           api_key: "2e8875a5be99cb8968af974e39ed9514",
13           language: "pt-BR",
14           page: 1,
15         }
16       });
17       console.log(response.data.results);
18     }
19
20     loadFilmes();
21
22   }, []);
23
24   return(
25     <div>
26       <h1>BEM VINDO A HOME</h1>
27     </div>
28   )
29 }
30
31 export default Home;
```

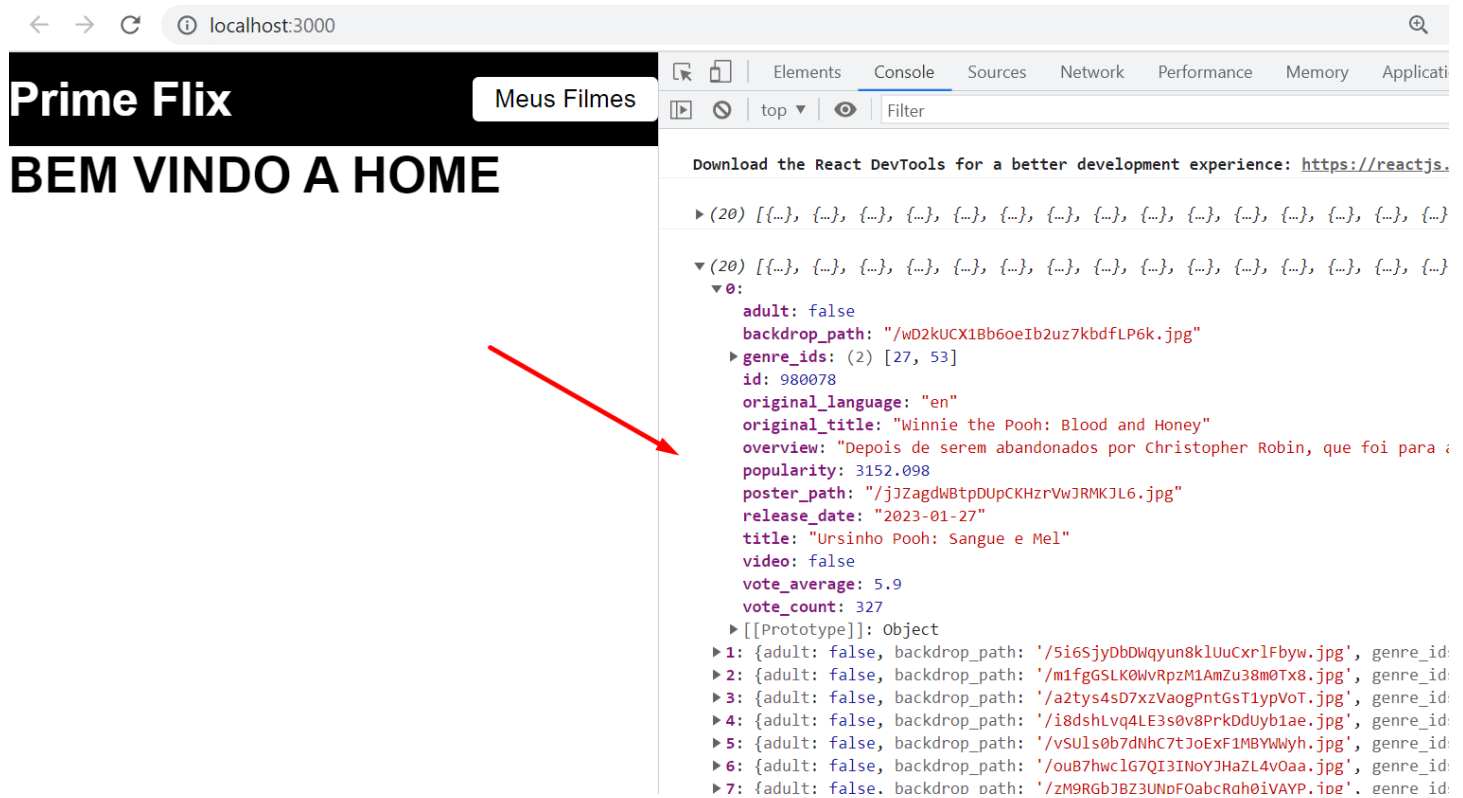
Line 6:20: 'setFilmes' is assigned a value but never used [no-unused-vars](#)

Search for the [keywords](#) to learn more about each warning.
To ignore, add `// eslint-disable-next-line` to the line before.

WARNING in [eslint]
src\pages\Home\index.js
Line 6:12: 'filmes' is assigned a value but never used [no-unused-vars](#)
Line 6:20: 'setFilmes' is assigned a value but never used [no-unused-vars](#)

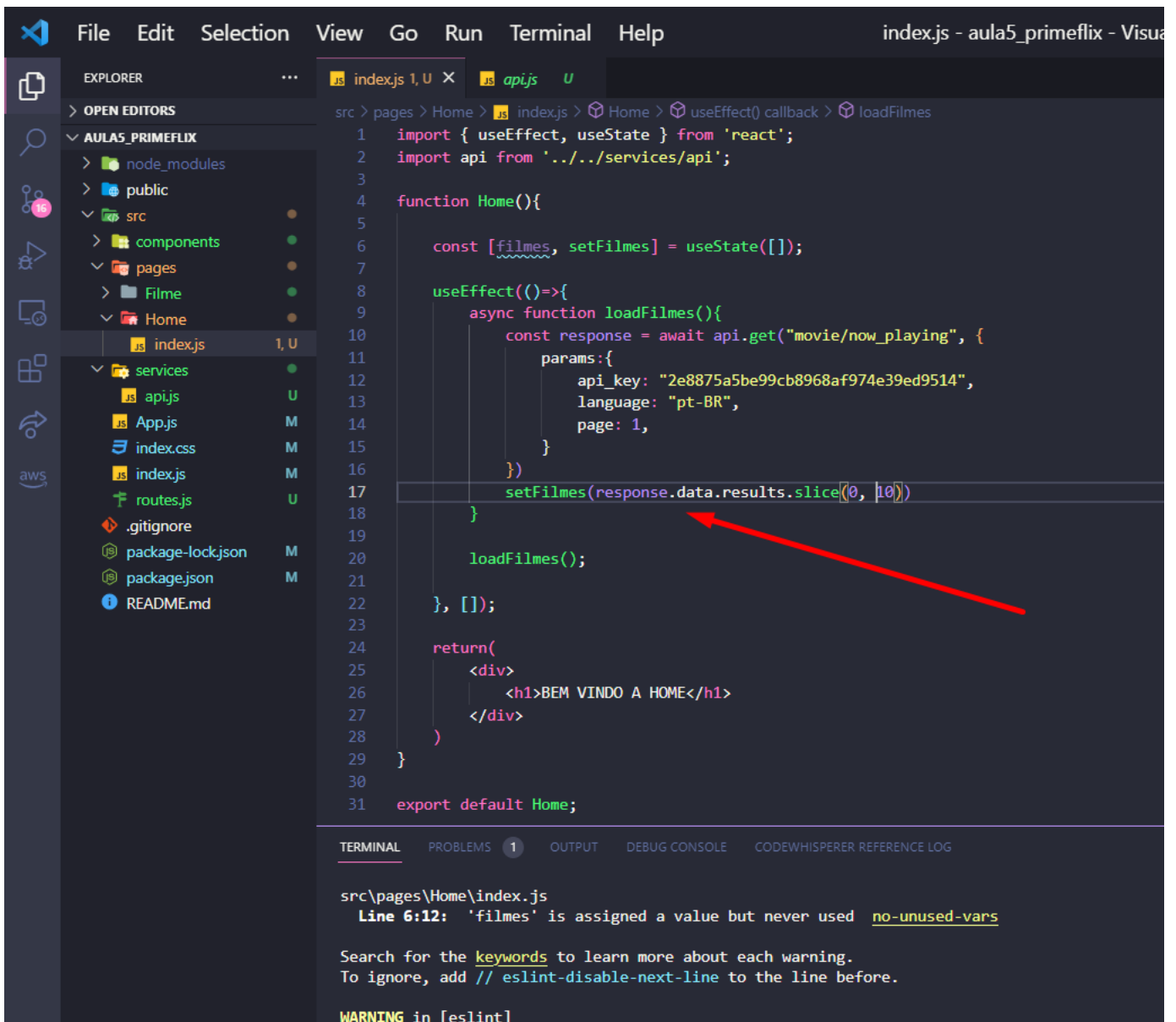
webpack compiled with 1 warning

41 – Agora podemos ver que apenas um array de filme é retornado dentro da nossa requisição da API, agora iremos consumir esse array e exibir os filmes na Home.



Listando Filmes em nossa aplicação utilizando requisição de API

42 – Não queremos esse array de filmes no console, queremos poder manipular as informações dentro da aplicação. Agora podemos usar o `useState`, utilize o `setFilmes` para receber o results da requisição da API, já aproveitamos para utilizar a função `slice` para exibir apenas os 10 primeiros resultados da API e não os 19.



```
1 import { useEffect, useState } from 'react';
2 import api from '../services/api';
3
4 function Home(){
5
6   const [filmes, setFilmes] = useState([]);
7
8   useEffect(()=>{
9     async function loadFilmes(){
10       const response = await api.get("movie/now_playing", {
11         params:{
12           api_key: "2e8875a5be99cb8968af974e39ed9514",
13           language: "pt-BR",
14           page: 1,
15         }
16       });
17       setFilmes(response.data.results.slice(0, 10));
18     }
19
20     loadFilmes();
21
22   }, []);
23
24   return(
25     <div>
26       <h1>BEM VINDO A HOME</h1>
27     </div>
28   )
29 }
30
31 export default Home;
```

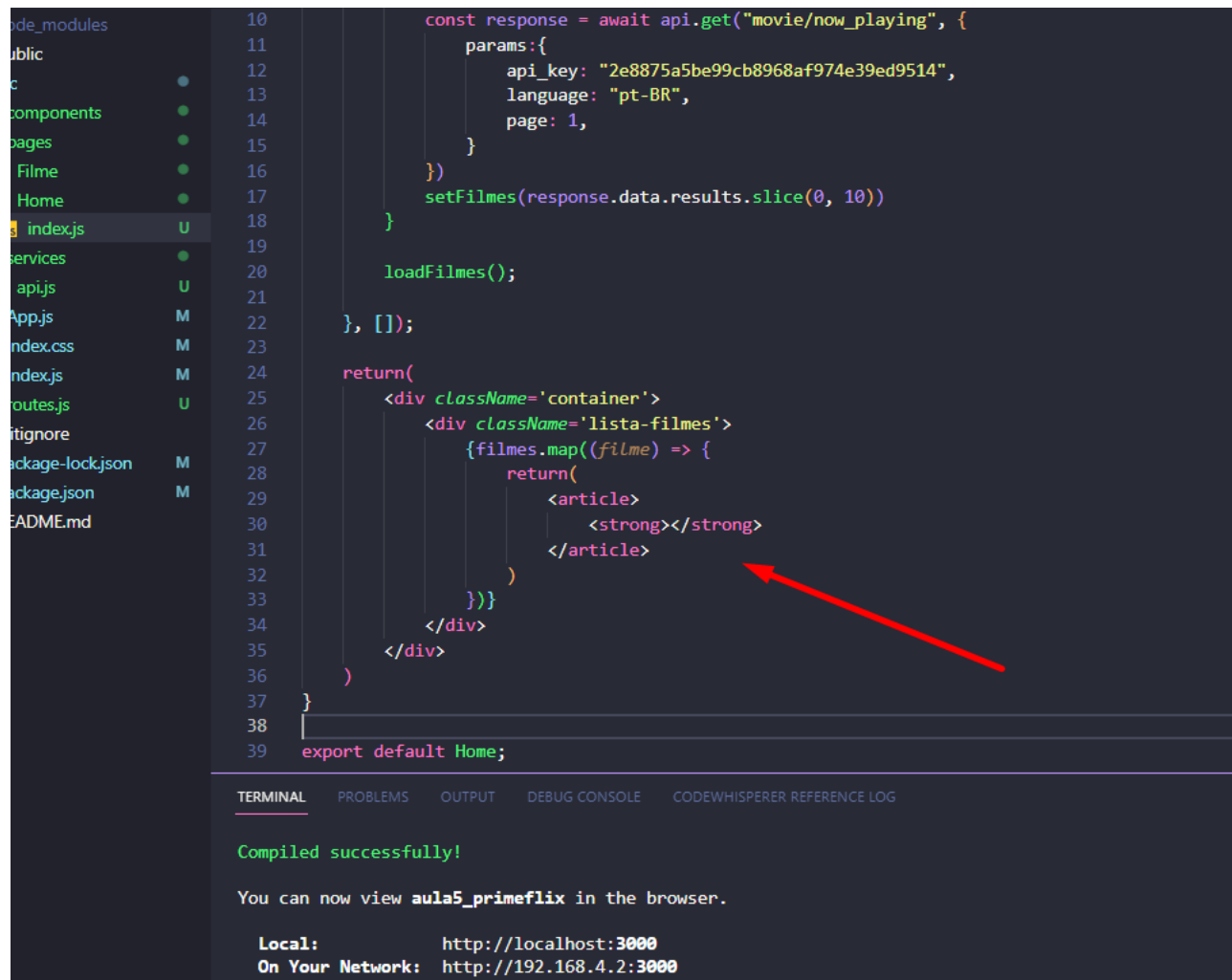
src\pages\Home\index.js

Line 6:12: 'filmes' is assigned a value but never used [no-unused-vars](#)

Search for the [keywords](#) to learn more about each warning.
To ignore, add `// eslint-disable-next-line` to the line before.

WARNING in [eslint]

43 – Agora vamos retornar a lista de filmes na Home da nossa aplicação. No lugar do `<h1>Bem vindo a Home</h1>` coloque uma div de container e dentro dela uma div que identificaremos como lista-filmes. Usaremos nossa `useState` `filmes` que recebe o array da API, como abaixo demonstrado:



The image shows a code editor with a file explorer on the left and a terminal at the bottom. The file explorer lists files like `index.js`, `App.js`, and `index.css`. The main editor displays a React component `Home` with the following code:

```
10 const response = await api.get("movie/now_playing", {
11   params: {
12     api_key: "2e8875a5be99cb8968af974e39ed9514",
13     language: "pt-BR",
14     page: 1,
15   }
16 })
17 setFilmes(response.data.results.slice(0, 10))
18 }
19
20 loadFilmes();
21
22 }, []);
23
24 return(
25   <div className='container'>
26     <div className='lista-filmes'>
27       {filmes.map((filme) => {
28         return(
29           <article>
30             <strong></strong>
31           </article>
32         )
33       })}
34     </div>
35   </div>
36 )
37 }
38
39 export default Home;
```

A red arrow points to the `<strong></strong>` placeholder in the `<article>` component. The terminal at the bottom shows the following output:

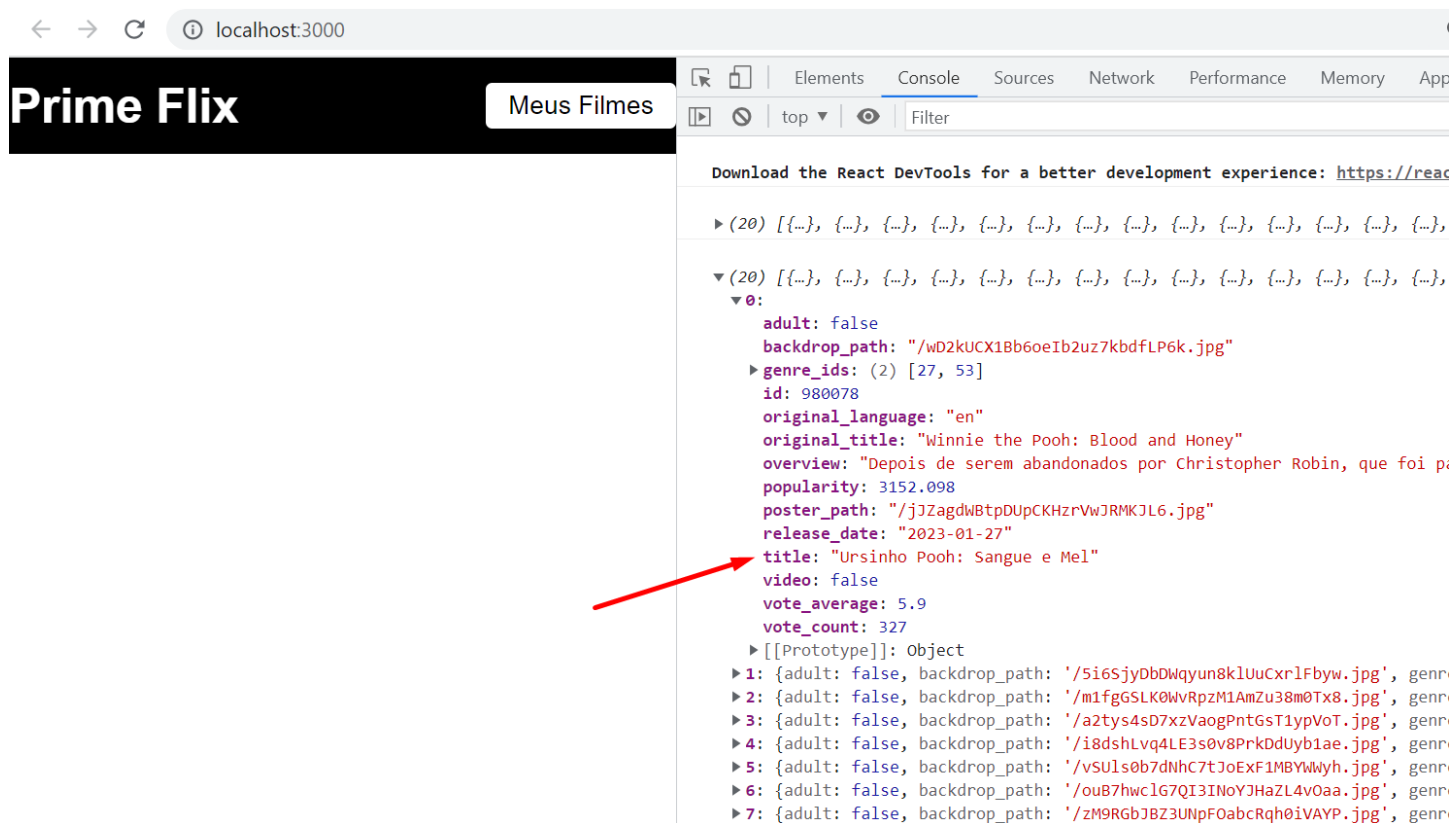
```
TERMINAL  PROBLEMS  OUTPUT  DEBUG CONSOLE  CODEWHISPERER REFERENCE LOG

Compiled successfully!

You can now view aula5_primeflix in the browser.

Local:      http://localhost:3000
On Your Network: http://192.168.4.2:3000
```

44 – Veja que no JSON de retorno da API temos diversas chaves (os nomes em roxo), como o `useState` filmes recebe todo esse JSON, podemos buscar cada informação individualmente, por exemplo, se eu quiser apenas pegar o título do filme posso usar o `filme.title` ou a popularidade `filme.popularity` e assim por diante.

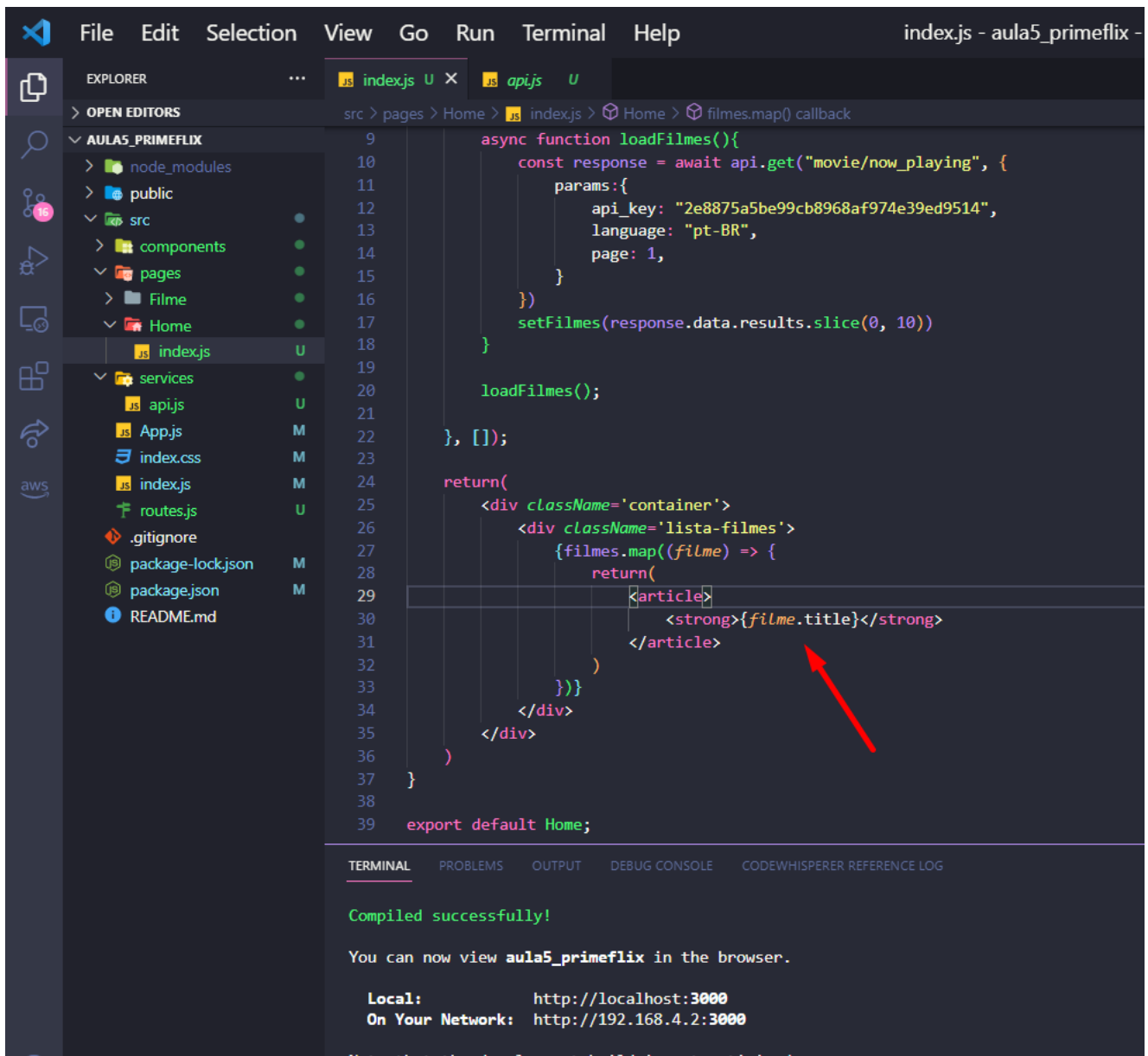


The screenshot shows a web browser at `localhost:3000` displaying the Prime Flix application. The application has a black header with the text "Prime Flix" and a button labeled "Meus Filmes". The React DevTools console is open, showing a JSON array of movie objects. A red arrow points to the `title` property of the first movie object, which is "Ursinho Pooh: Sangue e Mel".

```
Download the React DevTools for a better development experience: https://react.dev/learn/how-to-install-react-devtools

▶ (20) [{...}, {...}, {...}, {...}, {...}, {...}, {...}, {...}, {...}, {...}, {...}, {...}, {...}, {...}, {...}, {...}, {...}, {...}, {...}, {...}]
▼ (20) [{...}, {...}, {...}, {...}, {...}, {...}, {...}, {...}, {...}, {...}, {...}, {...}, {...}, {...}, {...}, {...}, {...}, {...}, {...}, {...}]
  ▼ 0:
    adult: false
    backdrop_path: "/wD2kUCX1Bb6oeIb2uz7kbfLP6k.jpg"
    ▶ genre_ids: (2) [27, 53]
    id: 980078
    original_language: "en"
    original_title: "Winnie the Pooh: Blood and Honey"
    overview: "Depois de serem abandonados por Christopher Robin, que foi p..."
    popularity: 3152.098
    poster_path: "/jjZagdwBtpDUpCKHzrVwJRMKJL6.jpg"
    release_date: "2023-01-27"
    title: "Ursinho Pooh: Sangue e Mel"
    video: false
    vote_average: 5.9
    vote_count: 327
    ▶ [[Prototype]]: Object
  ▶ 1: {adult: false, backdrop_path: '/5i6SjyDbbWqyun8k1UuCxrlFbyw.jpg', genre...}
  ▶ 2: {adult: false, backdrop_path: '/m1fgGSLK0WvRpzM1AmZu38m0Tx8.jpg', genre...}
  ▶ 3: {adult: false, backdrop_path: '/a2tys4sD7xzVaogPntGsT1ypVoT.jpg', genre...}
  ▶ 4: {adult: false, backdrop_path: '/i8dshLvq4LE3s0v8PrkDduyb1ae.jpg', genre...}
  ▶ 5: {adult: false, backdrop_path: '/vSULs0b7dNhC7tJoExF1MBYwMyh.jpg', genre...}
  ▶ 6: {adult: false, backdrop_path: '/ouB7hwc1G7QI3INoYJHaZL4v0aa.jpg', genre...}
  ▶ 7: {adult: false, backdrop_path: '/zM9RGBJJBZ3UNpFOabcRqh0iVAYP.jpg', genre...}
```

45 – Dentro da tag coloque {filme.title}, assim iremos pegar apenas os nomes de todo o nosso array de 10 filmes.



```
File Edit Selection View Go Run Terminal Help index.js - aula5_primeflix -

EXPLORER
OPEN EDITORS
src > pages > Home > index.js > Home > filmes.map() callback

AULA5_PRIMEFLIX
  node_modules
  public
  src
    components
    pages
    Filme
    Home
      index.js
    services
      api.js
      App.js
      index.css
      index.js
      routes.js
  .gitignore
  package-lock.json
  package.json
  README.md

9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39

async function loadFilmes(){
  const response = await api.get("movie/now_playing", {
    params:{
      api_key: "2e8875a5be99cb8968af974e39ed9514",
      language: "pt-BR",
      page: 1,
    }
  })
  setFilmes(response.data.results.slice(0, 10))
}

loadFilmes();

}, []);

return(
  <div className='container'>
    <div className='lista-filmes'>
      {filmes.map((filme) => {
        return(
          <article>
            <strong>{filme.title}</strong>
          </article>
        )
      })}
    </div>
  </div>
)

export default Home;
```

Compiled successfully!

You can now view **aula5_primeflix** in the browser.

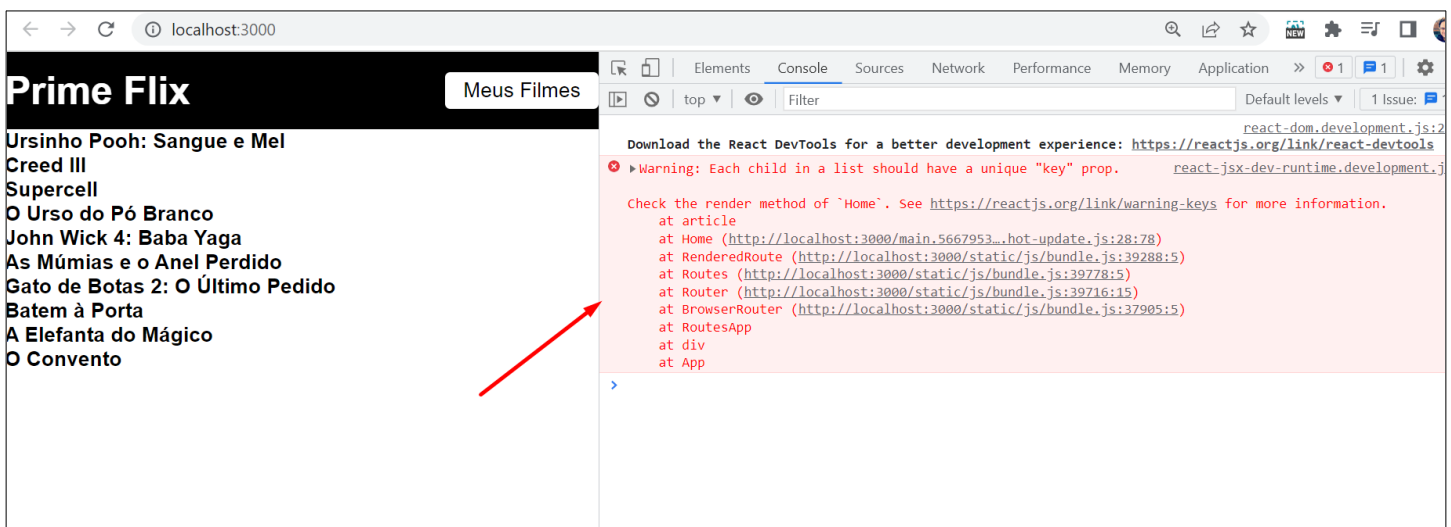
Local: http://localhost:3000
On Your Network: http://192.168.4.2:3000

Note that the development build is not optimized.

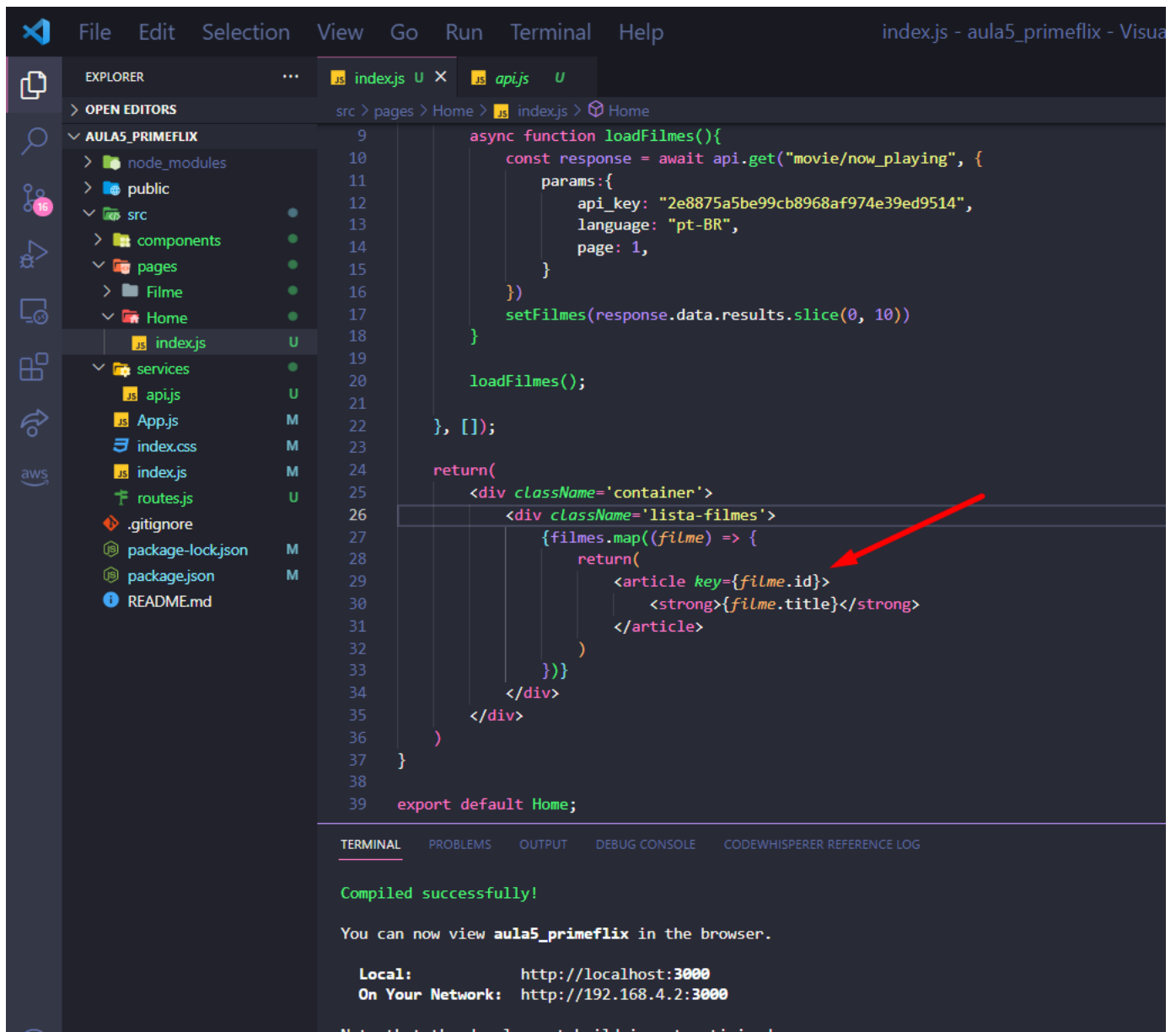
46 – Pronto, ele percorreu o array de 10 filmes e utilizando o map listou todos os filme na Home



47 – Mesmo mostrando a lista nosso console avisa que cada elemento da lista precisa ter uma chave única para funcionar corretamente. De fato na API podemos ver que cada filme usa um id para identificação, iremos utilizar esse mesmo id, assim no futuro podemos usar o id para rotear cada filme para uma página diferente.



48 – Dentro do <article> informe qual será a key que cada elemento criado receberá.



```
9      async function loadFilmes(){
10        const response = await api.get("movie/now_playing", {
11          params:{
12            api_key: "2e8875a5be99cb8968af974e39ed9514",
13            language: "pt-BR",
14            page: 1,
15          }
16        })
17        setFilmes(response.data.results.slice(0, 10))
18      }
19
20      loadFilmes();
21
22    }, []);
23
24    return(
25      <div className='container'>
26        <div className='lista-filmes'>
27          {filmes.map((filme) => {
28            return(
29              <article key={filme.id}>
30                <strong>{filme.title}</strong>
31              </article>
32            )
33          })}
34        </div>
35      </div>
36    )
37  }
38
39  export default Home;
```

TERMINAL PROBLEMS OUTPUT DEBUG CONSOLE CODEWHISPERER REFERENCE LOG

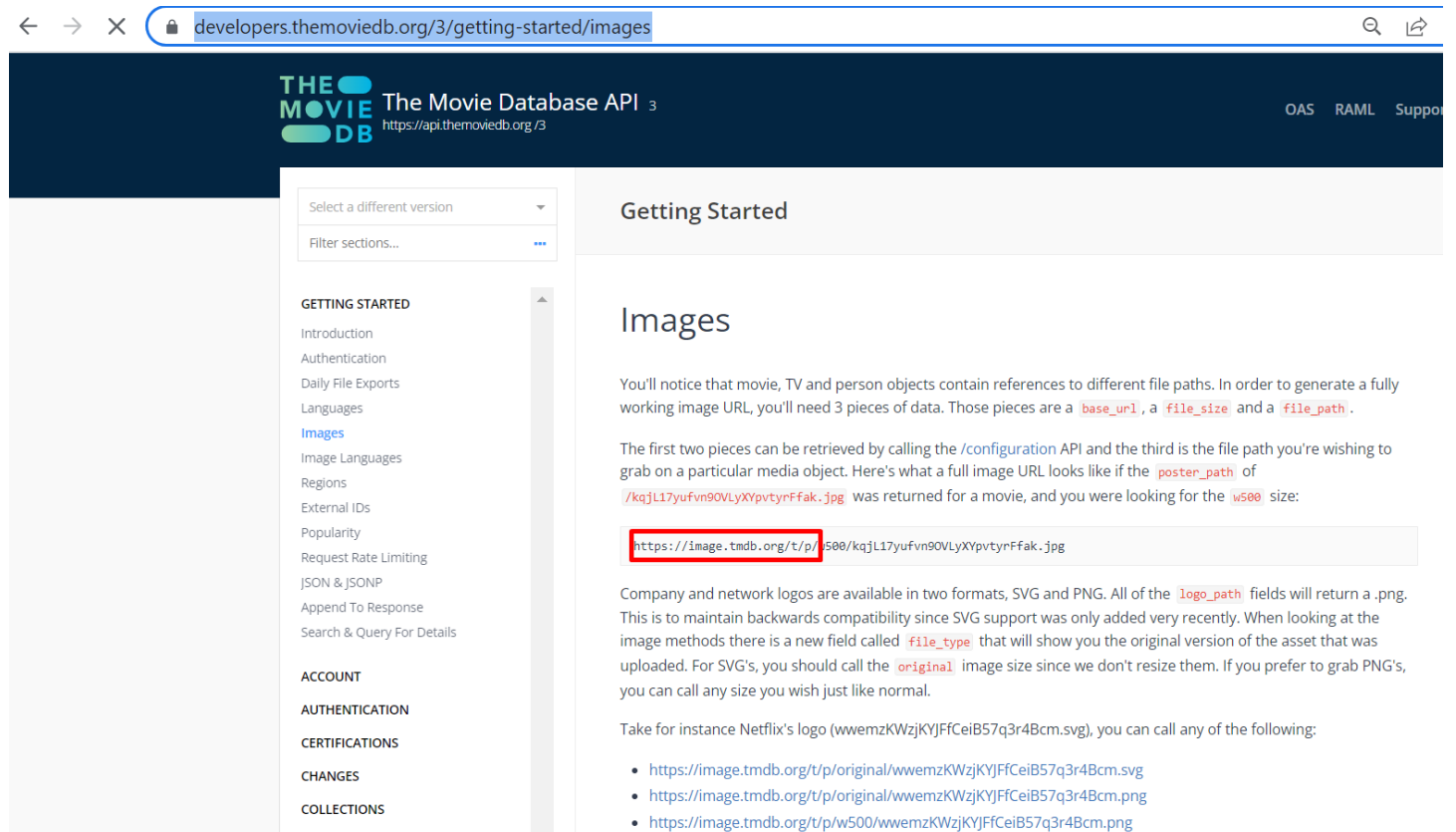
Compiled successfully!

You can now view **aula5_primeflix** in the browser.

Local: http://localhost:3000
On Your Network: http://192.168.4.2:3000

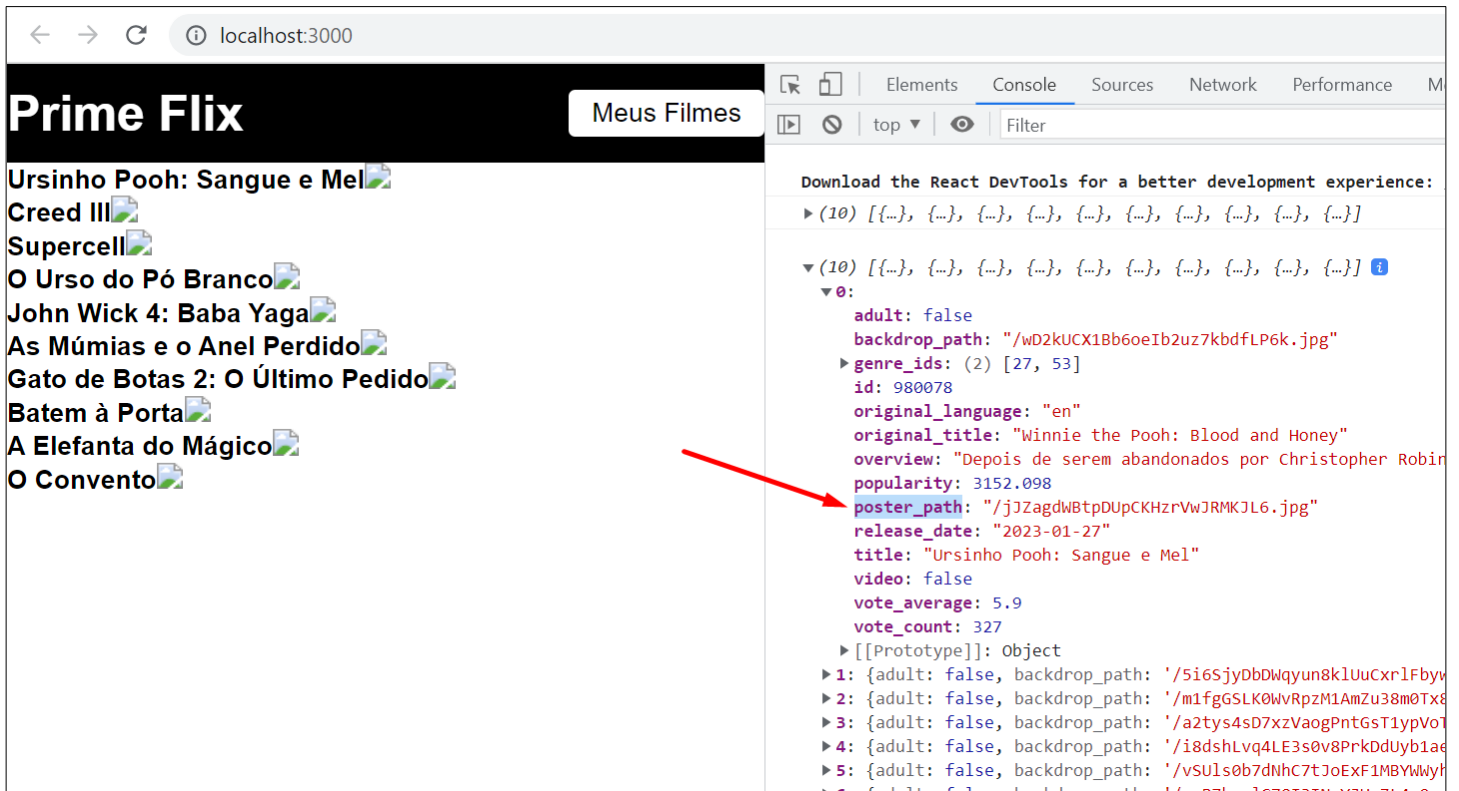
Note that the development build is not optimized.

49 – Agora queremos mostrar a imagem de cada filme que a API nos fornece. Para isso precisamos recorrer a documentação e entender como a API funciona para tratamento da imagem: <https://developers.themoviedb.org/3/getting-started/images>. Perceba que toda imagem tem uma URL base e depois o restante depende de cada imagem, precisamos codificar essa regra em nossa aplicação.



The screenshot shows the 'Getting Started' section of the The Movie Database API documentation. The left sidebar contains a navigation menu with categories like GETTING STARTED, ACCOUNT, AUTHENTICATION, CERTIFICATIONS, CHANGES, and COLLECTIONS. The 'Images' section is highlighted under GETTING STARTED. The main content area is titled 'Images' and explains how to construct a full image URL from the API response. It states that movie, TV, and person objects contain references to different file paths, and to generate a fully working image URL, you need three pieces of data: `base_url`, `file_size`, and `file_path`. The first two pieces can be retrieved by calling the `/configuration` API, and the third is the file path you're wishing to grab on a particular media object. An example shows a `poster_path` of `/kqjL17yufvn90VLyXVpvtYrFfak.jpg` was returned for a movie, and you were looking for the `w500` size. A code block shows the resulting URL: `https://image.tmdb.org/t/p/w500/kqjL17yufvn90VLyXVpvtYrFfak.jpg`. The text explains that company and network logos are available in two formats, SVG and PNG, and that the `logo_path` fields will return a .png. It also mentions a new field called `file_type` that will show you the original version of the asset that was uploaded. For SVG's, you should call the `original` image size since we don't resize them. If you prefer to grab PNG's, you can call any size you wish just like normal. A note provides an example for Netflix's logo (wwemzKWzjKYJFfCeIB57q3r4Bcm.svg) and lists three example URLs: `https://image.tmdb.org/t/p/original/wwemzKWzjKYJFfCeIB57q3r4Bcm.svg`, `https://image.tmdb.org/t/p/original/wwemzKWzjKYJFfCeIB57q3r4Bcm.png`, and `https://image.tmdb.org/t/p/w500/wwemzKWzjKYJFfCeIB57q3r4Bcm.png`.

50 – Perceba que no retorno da API a chave `poster_path` só contém o final da URL da imagem, precisamos então concatenar ambas para que a imagem de cada filme possa parecer corretamente.



The screenshot shows a web browser at `localhost:3000` displaying a web application titled "Prime Flix". The application has a navigation bar with a "Meus Filmes" button. Below the navigation bar, there is a list of movies, each with a title and a small image icon:

- Ursinho Pooh: Sangue e Mel
- Creed III
- Supercell
- O Urso do Pó Branco
- John Wick 4: Baba Yaga
- As Múmias e o Anel Perdido
- Gato de Botas 2: O Último Pedido
- Batem à Porta
- A Elefanta do Mágico
- O Convento

On the right side of the browser, the React DevTools console is open, showing the "Console" tab. The console displays a message about downloading React DevTools, followed by a list of 10 objects. The first object is expanded, showing the following properties:

- `adult`: false
- `backdrop_path`: `"/wD2kUCX18b6oeIb2uz7kdbfLP6k.jpg"`
- `genre_ids`: (2) [27, 53]
- `id`: 980078
- `original_language`: "en"
- `original_title`: "Winnie the Pooh: Blood and Honey"
- `overview`: "Depois de serem abandonados por Christopher Robin"
- `popularity`: 3152.098
- `poster_path`: `"/jJZagdwBtpDUpCKHzrVwJRMKJL6.jpg"`
- `release_date`: "2023-01-27"
- `title`: "Ursinho Pooh: Sangue e Mel"
- `video`: false
- `vote_average`: 5.9
- `vote_count`: 327

A red arrow points from the `poster_path` property in the console to the "Ursinho Pooh: Sangue e Mel" movie entry in the application's list.

51 – Adicione a tag `img` e como o caminho precisa ser concatenado, ao invés de usar o `' '` abre e fecha aspas simples, vamos usar o template string (`` ``) que permite concatenação. Ele tem esse formato, no teclado fica ao lado da letra P, pressione a tecla shift esquerdo e depois o ```

```
24
25   return(
26     <div className='container'>
27       <div className='lista-filmes'>
28         {filmes.map((filme) => {
29           return(
30             <article key={filme.id}>
31               <strong>{filme.title}</strong>
32               <img src={``} />
33             </article>
34           )
35         })}
36       </div>
37     </div>
38   )
39 }
40
41 export default Home;
```



52 – Dessa forma conseguimos chamar concatenar corretamente a URL. Não se preocupe se a imagem estiver enorme, iremos estilizar a página.

```
File Edit Selection View Go Run Terminal Help index.js - aula5_primeflix - Visual Studio Code

EXPLORER
src > pages > Home > index.js
AULAS_PRIMEFLIX
  node_modules
  public
  src
    components
    pages
    Filme
    Home
      index.js
  services
    apijs
    App.js
    index.css
    index.js
    routes.js
  .gitignore
  package-lock.json
  package.json
  README.md

index.js
17     setFilmes(response.data.results.slice(0, 10))
18     console.log(response.data.results.slice(0, 10))
19   }
20
21   loadFilmes();
22
23 }, []);
24
25 return(
26   <div className='container'>
27     <div className='lista-filmes'>
28       {filmes.map((filme) => {
29         return(
30           <article key={filme.id}>
31             <strong>{filme.title}</strong>
32             <img src={`https://image.tmdb.org/t/p/original${filme.poster_path}`} alt={filme.title} />
33           </article>
34         )
35       })}
36     </div>
37   </div>
38 )
39 }
40
41 export default Home;
```

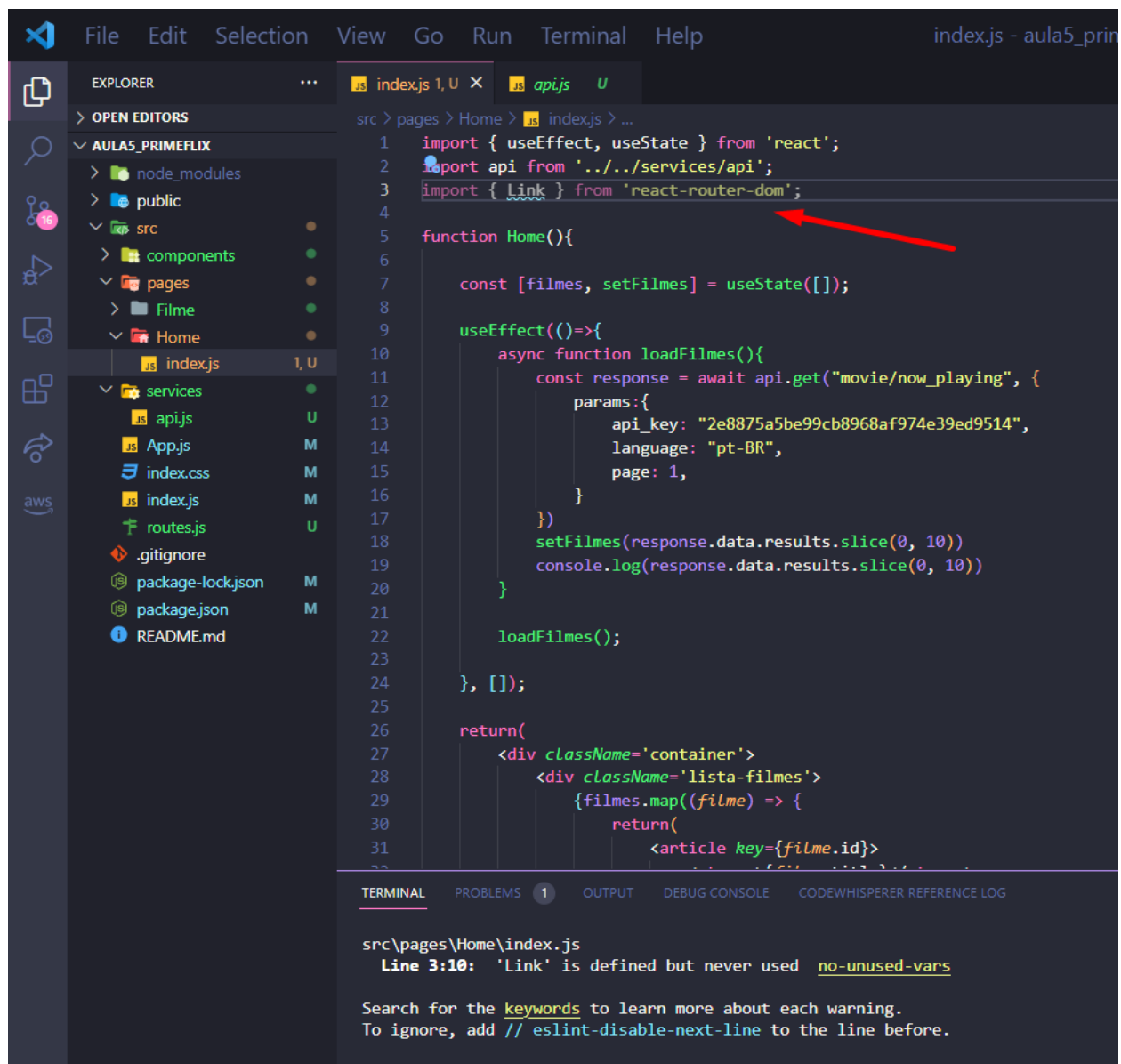
TERMINAL PROBLEMS OUTPUT DEBUG CONSOLE CODEWHISPERER REFERENCE LOG

Compiled successfully!

You can now view **aula5_primeflix** in the browser.

Local: http://localhost:3000
On Your Network: http://192.168.4.2:3000

53 – Vamos agora configurar as rotas de cada filme. Adicione no começo do index.js do Home o Link.

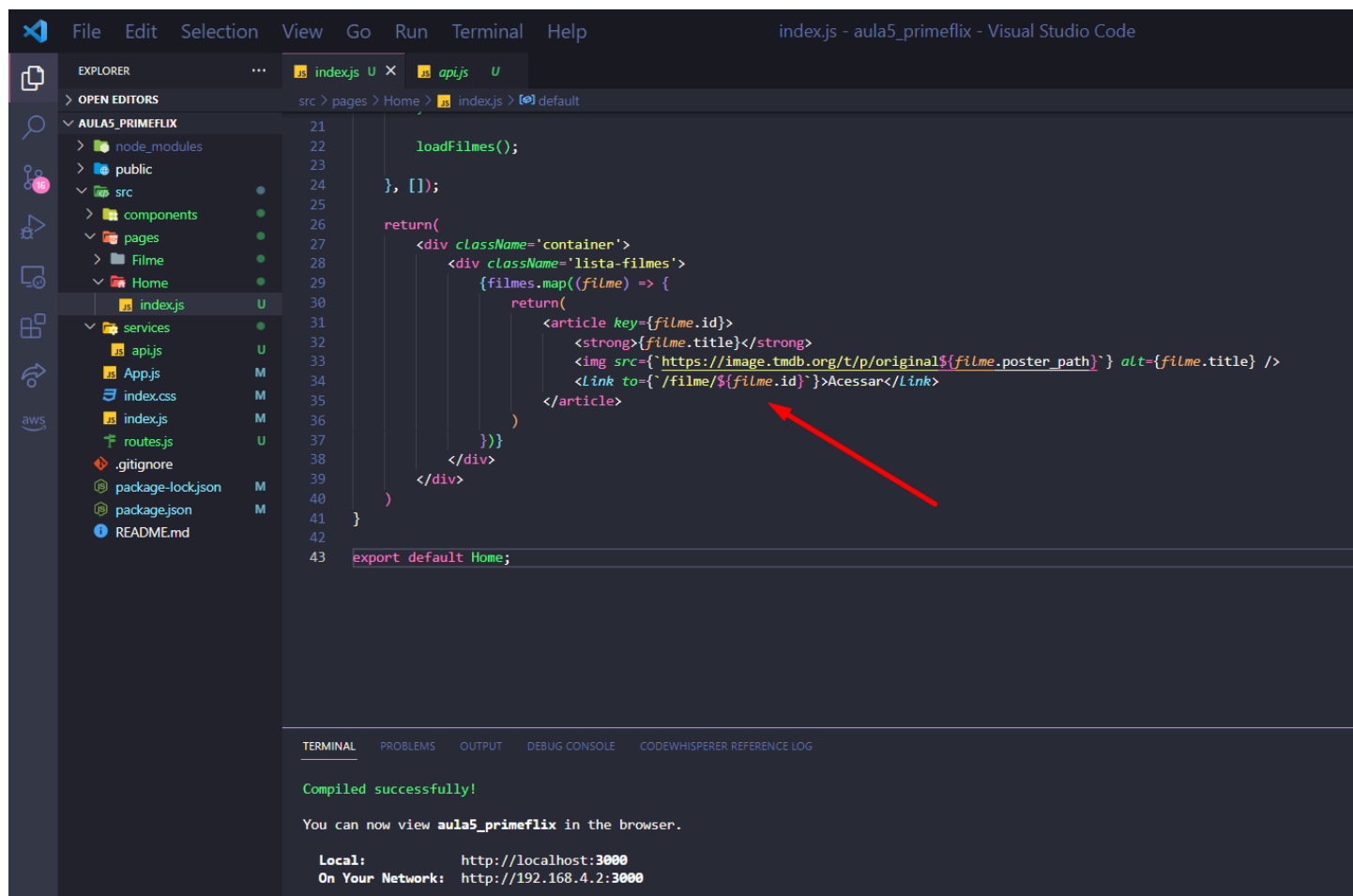


```
1 import { useEffect, useState } from 'react';
2 import api from '../services/api';
3 import { Link } from 'react-router-dom';
4
5 function Home(){
6
7   const [filmes, setFilmes] = useState([]);
8
9   useEffect(()=>{
10     async function loadFilmes(){
11       const response = await api.get("movie/now_playing", {
12         params:{
13           api_key: "2e8875a5be99cb8968af974e39ed9514",
14           language: "pt-BR",
15           page: 1,
16         }
17       })
18       setFilmes(response.data.results.slice(0, 10))
19       console.log(response.data.results.slice(0, 10))
20     }
21
22     loadFilmes();
23
24   }, []);
25
26   return(
27     <div className='container'>
28       <div className='lista-filmes'>
29         {filmes.map((filme) => {
30           return(
31             <article key={filme.id}>
```

src\pages\Home\index.js
Line 3:10: 'Link' is defined but never used [no-unused-vars](#)

Search for the [keywords](#) to learn more about each warning.
To ignore, add `// eslint-disable-next-line` to the line before.

54 – Agora vamos criar um link que quando clicado irá abrir uma nova página tendo como rota /filme/o id de cada filme:



```
File Edit Selection View Go Run Terminal Help index.js - aula5_primeflix - Visual Studio Code

EXPLORER
src > pages > Home > index.js [e] default

AULAS_PRIMEFLIX
├── node_modules
├── public
├── src
│   ├── components
│   ├── pages
│   │   ├── Filme
│   │   └── Home
│       ├── index.js
│       ├── services
│       │   ├── api.js
│       │   ├── App.js
│       │   ├── index.css
│       │   ├── index.js
│       │   ├── routes.js
│       │   ├── .gitignore
│       │   ├── package-lock.json
│       │   ├── package.json
│       │   └── README.md
└── ...

index.js
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43

loadFilmes();

}, []);

return(
  <div className='container'>
    <div className='lista-filmes'>
      {filmes.map((filme) => {
        return(
          <article key={filme.id}>
            <strong>{filme.title}</strong>
            <img src={`https://image.tmdb.org/t/p/original${filme.poster_path}`} alt={filme.title} />
            <Link to={`/filme/${filme.id}`}>Acessar</Link>
          </article>
        )
      })}
    </div>
  </div>
)

export default Home;
```

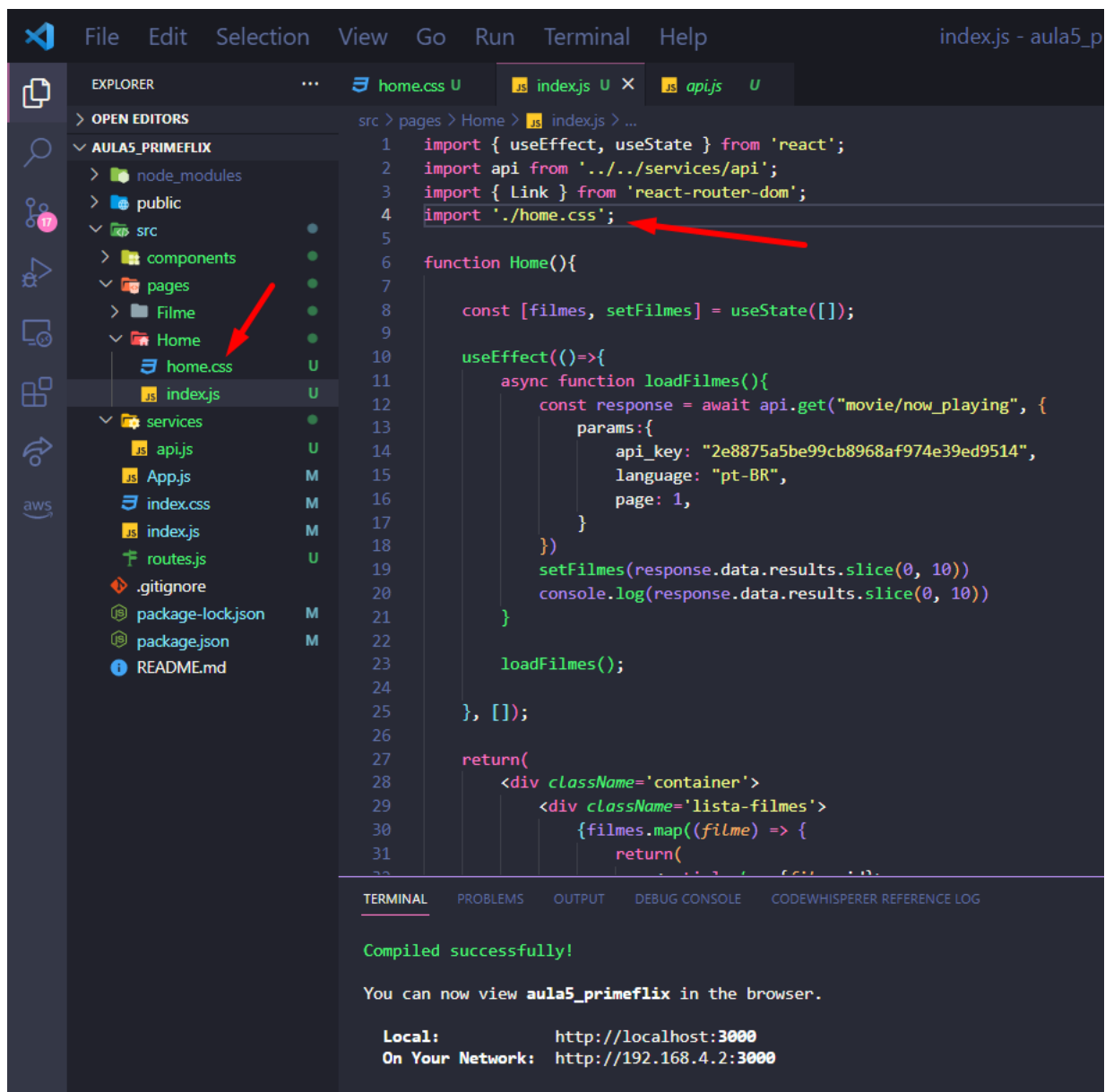
TERMINAL PROBLEMS OUTPUT DEBUG CONSOLE CODEWHISPERER REFERENCE LOG

Compiled successfully!

You can now view **aula5_primeflix** in the browser.

Local: http://localhost:3000
On Your Network: http://192.168.4.2:3000

55 – Dentro da pasta Home crie um arquivo chamado home.css e já faça a importação desse arquivo dentro de index.js.



The screenshot shows the Visual Studio Code interface. On the left, the Explorer sidebar shows the project structure with the 'Home' folder selected under 'pages'. The 'home.css' file is highlighted. The main editor shows the 'index.js' file with the following code:

```
1 import { useEffect, useState } from 'react';
2 import api from '../services/api';
3 import { Link } from 'react-router-dom';
4 import './home.css';
5
6 function Home(){
7
8   const [filmes, setFilmes] = useState([]);
9
10  useEffect(()=>{
11    async function loadFilmes(){
12      const response = await api.get("movie/now_playing", {
13        params:{
14          api_key: "2e8875a5be99cb8968af974e39ed9514",
15          language: "pt-BR",
16          page: 1,
17        }
18      });
19      setFilmes(response.data.results.slice(0, 10))
20      console.log(response.data.results.slice(0, 10))
21    }
22    loadFilmes();
23  }, []);
24
25  return(
26    <div className='container'>
27      <div className='lista-filmes'>
28        {filmes.map((filme) => {
29          return(
30            <div key={filme.id}>
31              <img alt={filme.poster_path}>
32              <h3>{filme.title}</h3>
33            </div>
34          )
35        })}
36      </div>
37    </div>
38  );
39}
```

Two red arrows point to the import statement on line 4 and the 'home.css' file in the Explorer. The bottom panel shows the terminal output:

```
Compiled successfully!

You can now view aula5_primeflix in the browser.

Local:      http://localhost:3000
On Your Network:  http://192.168.4.2:3000
```

56 – home.css

```
src > pages > Home > home.css > .lista-filmes a
1
2  .lista-filmes{
3      max-width: 800px;
4      margin: 14px auto;
5  }
6
7  .lista-filmes article{
8      width: 100%;
9      background-color: white;
10     padding: 15px;
11     border-radius: 4px;
12 }
13
14 .lista-filmes strong{
15     margin-bottom: 14px;
16     text-align: center;
17     font-size: 22px;
18     display: block;
19 }
20
21 .lista-filmes img{
22     width: 900px;
23     max-width: 100%;
24     max-height: 340px;
25     object-fit: cover;
26     display: block;
27     border-top-left-radius: 8px;
28     border-top-right-radius: 8px;
29 }
30
31 .lista-filmes a{
32     display: flex;
33     align-items: center;
34     justify-content: center;
35     padding: 10px 0;
36     font-size: 24px;
37     background-color: #116FE8;
38     text-decoration: none;
39     color: white;
40     border-bottom-left-radius: 8px;
41     border-bottom-right-radius: 8px;
42 }
```

57 – Pronto, temos nossa Home sendo renderizada com base em uma requisição de API de modo automático :D

